AN NAJAH NATIONAL UNIVERSITY | CYBER SECURITY

System Administration Practical Report

Lectures 1 to 4

Student: Ahmad Nassar
Course: System and Network Administration
Practical session: 13 October 2026
Platform: Ubuntu in VMware

Purpose and scope

I used an Ubuntu virtual machine to practise administration tasks, Linux commands and Bash shell features. This report presents the commands I used, the results I observed and screenshots from my practical session. It covers user accounts, backups, software installation, monitoring, networking, a local web service, permissions, documentation, file operations and shell behaviour.

The supplied Word document contained 59 screenshots in reverse chronological order. I selected the useful evidence and organised it by lecture and task. Short launch-only captures and repeated documentation captures were omitted. The figures preserve the original terminal and file-manager evidence.

Part A contains the recorded practical work. Part B adds 26 educational simulations with labelled illustrations, procedures and expected results for the remaining topics. These model results are not additional execution on my VM.

Main outcome

The recorded tests include successful backup restoration with matching file checksums, actual cron executions, successful account creation and removal, network connectivity, an HTTP 200 response and an access-denied permission test. The final cron configuration schedules a backup daily at 18:00 in the guest time zone.

Installation coverage

The actual screenshots show an already installed Ubuntu system. A new installation, partitioning and boot-loader configuration are covered by labelled model applications in Part B, including installer choices, storage, GRUB, network boot and automated deployment.

Report evidence	Interpretation
Executed and verified	A screenshot shows the command and an observable result.
Configured	A setting is shown, but its later scheduled outcome may not be recorded.
Simulation	An educational model illustrates steps and expected results.
Not verified by execution	No suitable student execution evidence was supplied.

REPORT GUIDE

Organisation and evidence coverage

The report follows the four lecture files. Within each experiment, the objective explains the purpose, the command list identifies the main actions, and the result states what the screenshot supports. Commands are shown as readable text as well as in the terminal evidence.

Lecture	Recorded practical work	Coverage
1 System administration	Hardware inventory; user accounts; backups and cron; software installation; monitoring; network and HTTP tests; permissions; local documentation.	Substantial evidence; some broader duties remain untested.
2 OS installation	Installed OS identification; disk and filesystem inventory; ordinary users; service and log checks; time synchronisation; firewall inspection.	Partial; the installation process itself was not recorded.
3 Linux commands	Files and directories; paths; manuals; filters; paging; date and calendar; vi editing.	Most listed command groups are evidenced.
4 Shells	Bash identification; history; aliases; variables; export; startup files; operators; jobs and pipes; globbing; redirection; quoting; substitution.	Most listed shell features are evidenced.

Reading the evidence

Some screenshots contain corrected typing or authentication mistakes. I report the final observed outcome without treating earlier failed attempts as successful commands. Missing-file errors and the denied access by nobody were intentional tests. The system log also contains genuine warnings and user-session errors, which are reported rather than hidden.

Distribution adaptation

The installation lecture uses older Fedora and Red Hat examples. My practical platform is Ubuntu, so package installation uses APT and the system-wide Bash file is /etc/bash.bashrc. The report does not claim that Fedora-specific installation or deployment tools were executed.

Navigation

Part A contains the recorded results and its experiment index. Part B follows with a simulation index, 26 illustrated model activities and an updated coverage table. Actual screenshots have numbered Figure captions; generated illustrations have Simulation figure captions.

LECTURE 1 | EXPERIMENT 01

Hardware inventory

Objective. Identify the CPU and memory visible inside my virtual machine.

Commands used

```
lscpu | head -n 20
free -h
```

Result. The guest reports an x86_64 Intel Core i7-4810MQ processor and 8 logical CPUs. It has about 7.2 GiB of RAM and 4.0 GiB of swap. These values describe the virtual machine view of the hardware.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ lscpu | head -n 20
Architecture:                         x86_64
CPU op-mode(s):                       32-bit, 64-bit
Address sizes:                        45 bits physical, 48 bits virtual
Byte Order:                           Little Endian
CPU(s):                               8
On-line CPU(s) list:                  0-7
Vendor ID:                            GenuineIntel
Model name:                           Intel(R) Core(TM) i7-4810MQ CPU @ 2.80GHz
CPU family:                           6
Model:                                60
Thread(s) per core:                   1
Core(s) per socket:                   1
Socket(s):                            8
Stepping:                             3
BogoMIPS:                             5587.09
Flags:                                fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat ps
e36 clflush mmx fxsr sse sse2 ss syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon nopl xtopology tsc_re
liable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt t
sc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm pti ssbd ibrs ibpb stibp fsgsbase tsc_adju
st bmi1 avx2 smep bmi2 invpcid xsaveopt arat md_clear flush_l1d arch_capabilities
Hypervisor vendor:                    VMware
Virtualization type:                  full
L1d cache:                            256 KiB (8 instances)
L1i cache:                            256 KiB (8 instances)
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.2Gi       1.4Gi       4.6Gi        17Mi       1.5Gi       5.8Gi
Swap:          4.0Gi          0B       4.0Gi
```

Figure 1. CPU identification and guest memory inventory.

LECTURE 1 | EXPERIMENT 02

Create and test a user account

Objective. Create a separate ordinary user and verify its identity, home directory and login shell.

Commands used

```
sudo adduser sysadmin_lab01
id sysadmin_lab01
getent passwd sysadmin_lab01
su - sysadmin_lab01
whoami
id
pwd
exit
```

Result. The account was created with UID 1001, GID 1001, home /home/sysadmin_lab01 and shell /bin/bash. The recorded groups contain the account group and users, with no sudo group. A successful su login then reported the new username and home directory.

Initial password entry was retried after a warning and mismatch. The screenshot subsequently confirms password setup and a successful account login.

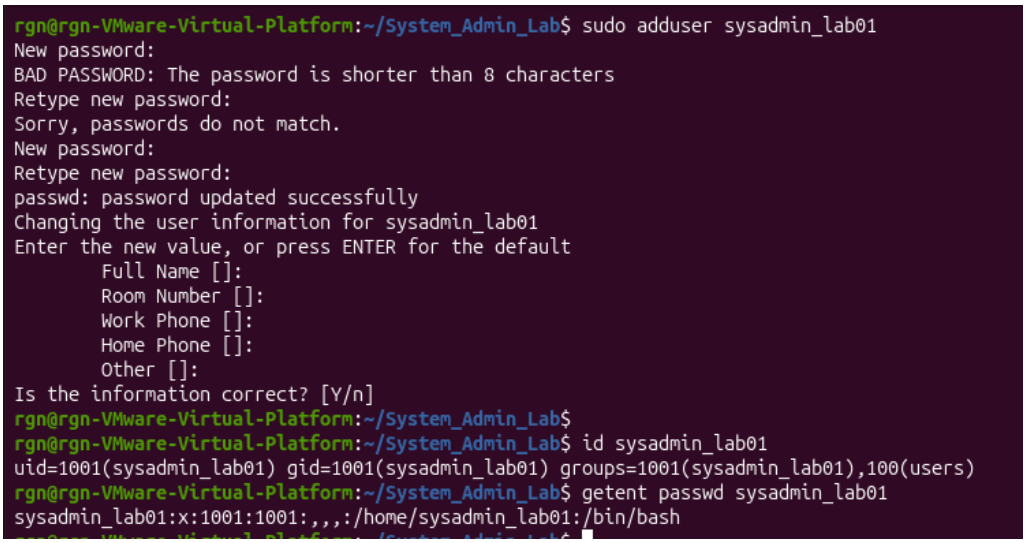


Figure 2. Account creation and account database verification.

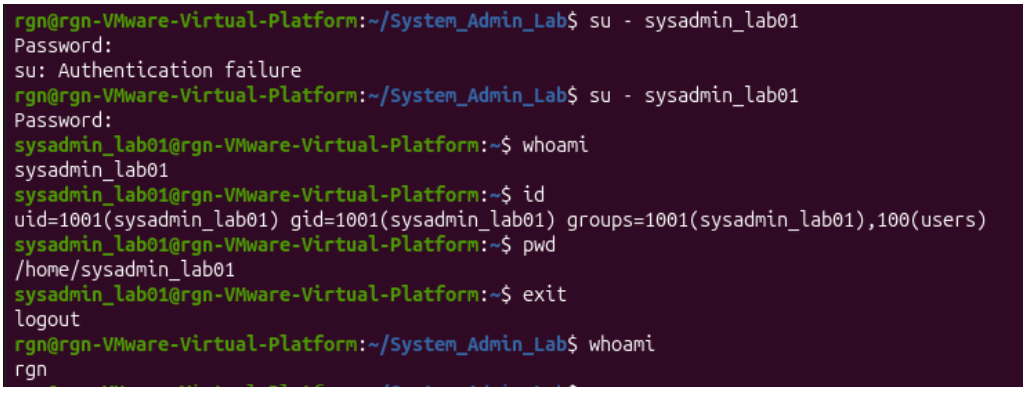


Figure 3. Successful login after an earlier failed authentication attempt.

LECTURE 1 | EXPERIMENT 03

Change a password and remove the lab account

Objective. Handle a password change, retest login and remove the temporary account and home directory.

Commands used

```
sudo passwd sysadmin_lab01
su - sysadmin_lab01
whoami
exit
sudo deluser --remove-home sysadmin_lab01
getent passwd sysadmin_lab01
test ! -d /home/sysadmin_lab01
```

Result. passwd reported that the password was updated successfully, and login as sysadmin_lab01 worked. After removal, the account lookup returned no entry and the directory test confirmed that the lab home was removed.

Earlier sudo authentication retries and a misspelled howami command were corrected before the final checks.

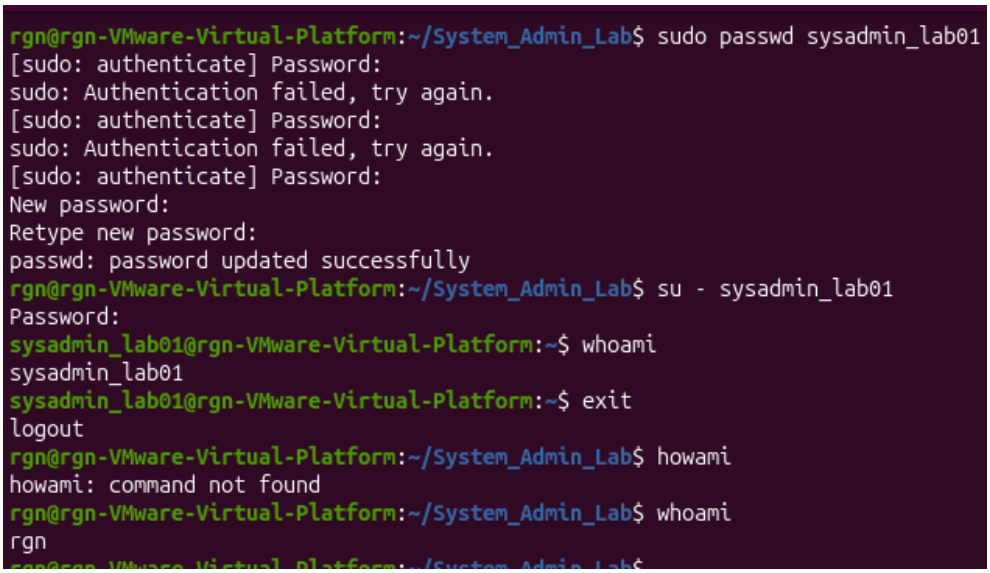


Figure 4. Successful password update and login; the parent username was checked again after exit.

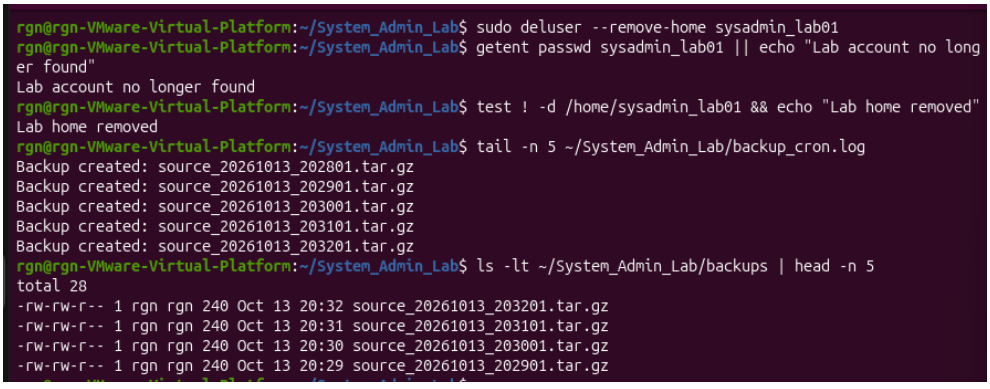


Figure 5. Account and home removal verified; the same capture also records cron backup executions.

LECTURE 1 | EXPERIMENT 04

Create and restore a backup

Objective. Archive the source directory, inspect the archive and verify restored file contents.

Commands used

```
mkdir -p backups restore
tar -czf backups/source_backup.tar.gz source
tar -tzf backups/source_backup.tar.gz
tar -xzf backups/source_backup.tar.gz -C restore
sha256sum source/notes.txt restore/source/notes.txt
```

Result. The archive included notes.txt, empty.txt and .hidden.txt. Extraction recreated the source directory under restore. The original and restored notes.txt files produced matching SHA-256 checksums, confirming identical contents.

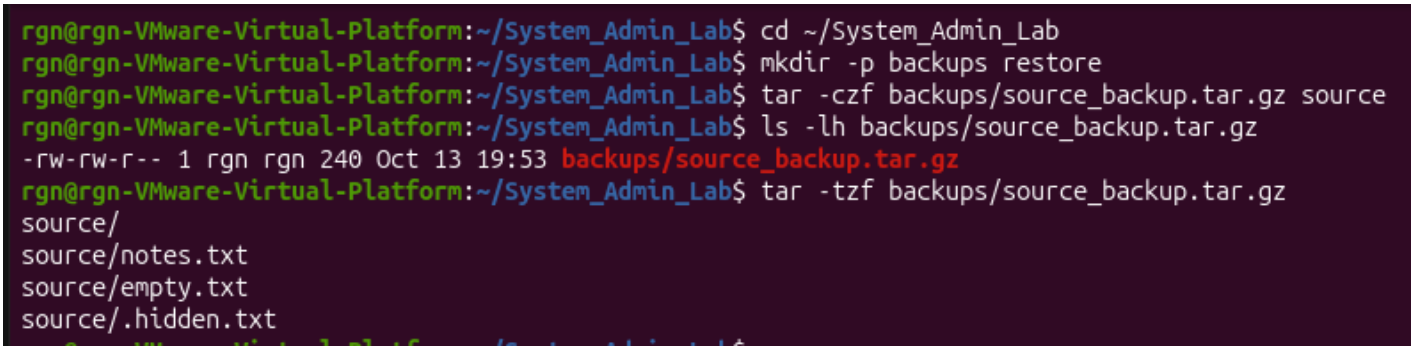


Figure 6. Backup archive creation and archive content listing.

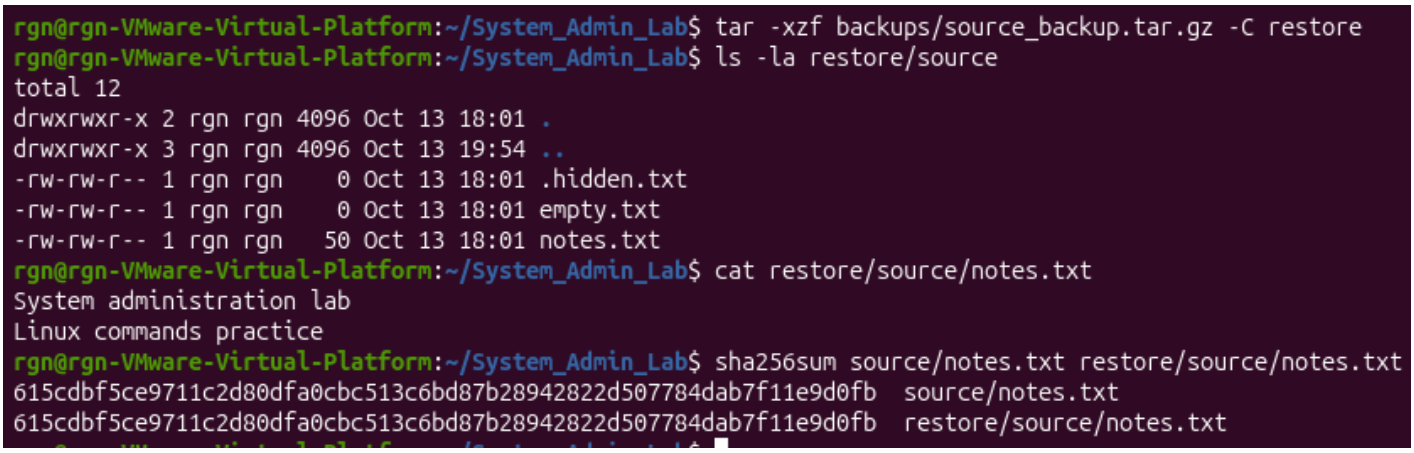


Figure 7. Restoration, readable recovered contents and matching SHA-256 checksums.

LECTURE 1 | EXPERIMENT 05

Automate backups with a Bash script

Objective. Create a reusable script that writes a timestamped archive and stops if an operation fails.

Commands used

```
#!/bin/bash
set -e
lab_base="$HOME/System_Admin_Lab"
lab_stamp=$(date +%Y%m%d_%H%M%S)
mkdir -p "$lab_base/backups"
tar -czf "$lab_base/backups/source_$lab_stamp.tar.gz" -C "$lab_base" source
echo "Backup created: source_$lab_stamp.tar.gz"
```

Result. After chmod +x backup_lab.sh, I ran ./backup_lab.sh successfully. It created source_20261013_195529.tar.gz and printed the archive name. The backup folder then contained both the manual archive and the timestamped archive.

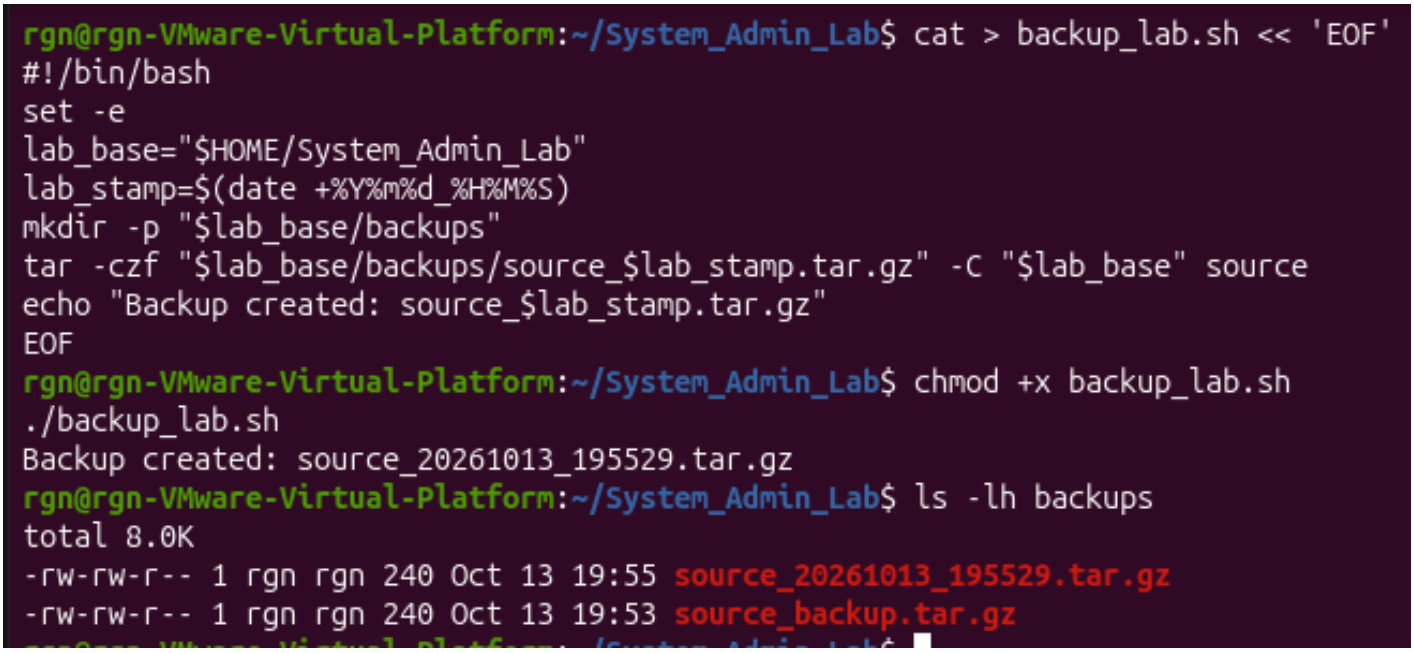


Figure 8. Script creation, executable permission and successful manual execution.

LECTURE 1 | EXPERIMENT 06

Verify the cron service

Objective. Confirm that the scheduling service is installed and active before adding a job.

Commands used

```
sudo apt install -y cron
sudo systemctl enable --now cron
systemctl is-active cron
crontab -l
```

Result. cron was already installed. The enable command completed and the service returned active. Before the test job was added, crontab -l reported no crontab for rgn.

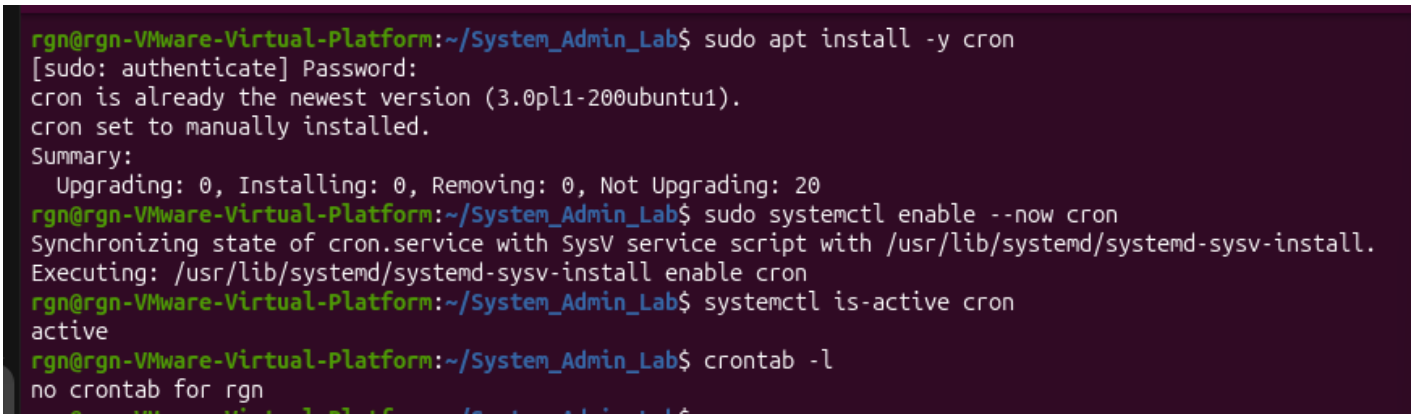


Figure 9. Installed cron package, active service and initially empty user crontab.

LECTURE 1 | EXPERIMENT 07

Test scheduled backups with cron

Objective. Verify the cron service and use a temporary every-minute schedule to observe real executions.

Commands used

```
crontab cron_lab.txt
crontab -l
tail -n 5 ~/System_Admin_Lab/backup_cron.log
ls -lt ~/System_Admin_Lab/backups | head -n 5
```

Result. cron was already installed and its service returned active. The every-minute job was installed in my user crontab. The log and archive listing show backup files at consecutive minute timestamps, including 20:28 through 20:32.

Test entry: * * * * * /home/rgn/System_Admin_Lab/backup_lab.sh >> /home/rgn/System_Admin_Lab/backup_cron.log 2>&1

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cd ~/System_Admin_Lab
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat > cron_lab.txt << 'EOF'
* * * * * /home/rgn/System_Admin_Lab/backup_lab.sh >> /home/rgn/System_Admin_Lab/backup_cron.log 2>&1
EOF
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ crontab cron_lab.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ crontab -l
* * * * * /home/rgn/System_Admin_Lab/backup_lab.sh >> /home/rgn/System_Admin_Lab/backup_cron.log 2>&1
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ crontab -l
* * * * * /home/rgn/System_Admin_Lab/backup_lab.sh >> /home/rgn/System_Admin_Lab/backup_cron.log 2>&1
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$
```

Figure 10. Temporary every-minute backup job installed.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ sudo deluser --remove-home sysadmin_lab01
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ getent passwd sysadmin_lab01 || echo "Lab account no long
er found"
Lab account no longer found
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ test ! -d /home/sysadmin_lab01 && echo "Lab home removed"
Lab home removed
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ tail -n 5 ~/System_Admin_Lab/backup_cron.log
Backup created: source_20261013_202801.tar.gz
Backup created: source_20261013_202901.tar.gz
Backup created: source_20261013_203001.tar.gz
Backup created: source_20261013_203101.tar.gz
Backup created: source_20261013_203201.tar.gz
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ls -lt ~/System_Admin_Lab/backups | head -n 5
total 28
-rw-rw-r-- 1 rgn rgn 240 Oct 13 20:32 source_20261013_203201.tar.gz
-rw-rw-r-- 1 rgn rgn 240 Oct 13 20:31 source_20261013_203101.tar.gz
-rw-rw-r-- 1 rgn rgn 240 Oct 13 20:30 source_20261013_203001.tar.gz
-rw-rw-r-- 1 rgn rgn 240 Oct 13 20:29 source_20261013_202901.tar.gz
```

Figure 11. Actual scheduled executions shown by log messages and timestamped archive files.

LECTURE 1 | EXPERIMENT 08

Set the final daily backup schedule

Objective. Replace the temporary test schedule with a daily backup at 18:00.

Commands used

```
crontab cron_daily.txt
crontab -l
date
```

Result. The final crontab contains the daily entry shown below. This confirms the configuration. The evidence of actual cron execution comes from the earlier every-minute test; a later daily execution was not captured.

Final entry: 0 18 * * * /home/rgn/System_Admin_Lab/backup_lab.sh >> /home/rgn/System_Admin_Lab/backup_cron.log 2>&1
The guest time zone is Asia/Hebron. The virtual machine and cron service must be running at the scheduled time.

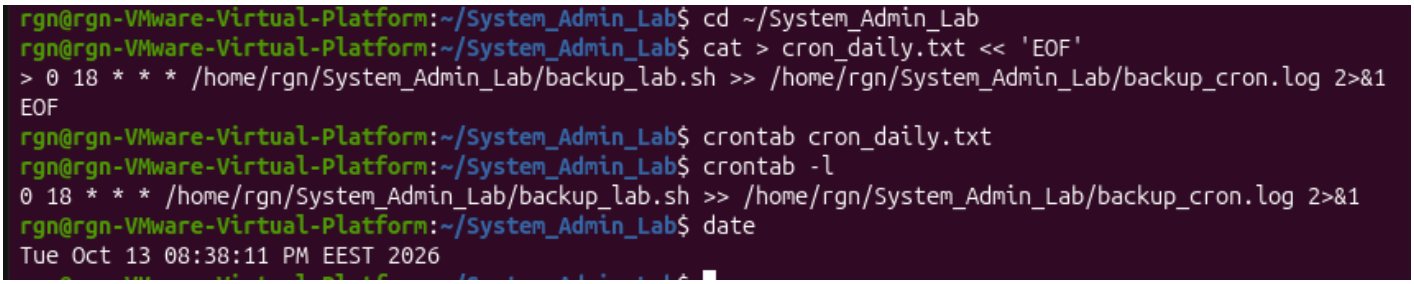


Figure 12. Final daily crontab entry and guest date and time.

LECTURE 1 | EXPERIMENT 09

Monitor resources and processes

Objective. Check system load, available memory, root filesystem space and processes ranked by CPU use.

Commands used

```
uptime
free -h
df -h /
ps -eo pid,user,comm,%cpu,%mem --sort=-%cpu | head -n 10
```

Result. The VM had been running for about 2 hours 46 minutes. The observed load averages were 0.00, 0.01 and 0.00. About 5.6 GiB of memory and 26 GB of root filesystem space were available. The process list identified gnome-shell as the highest CPU user in that snapshot.

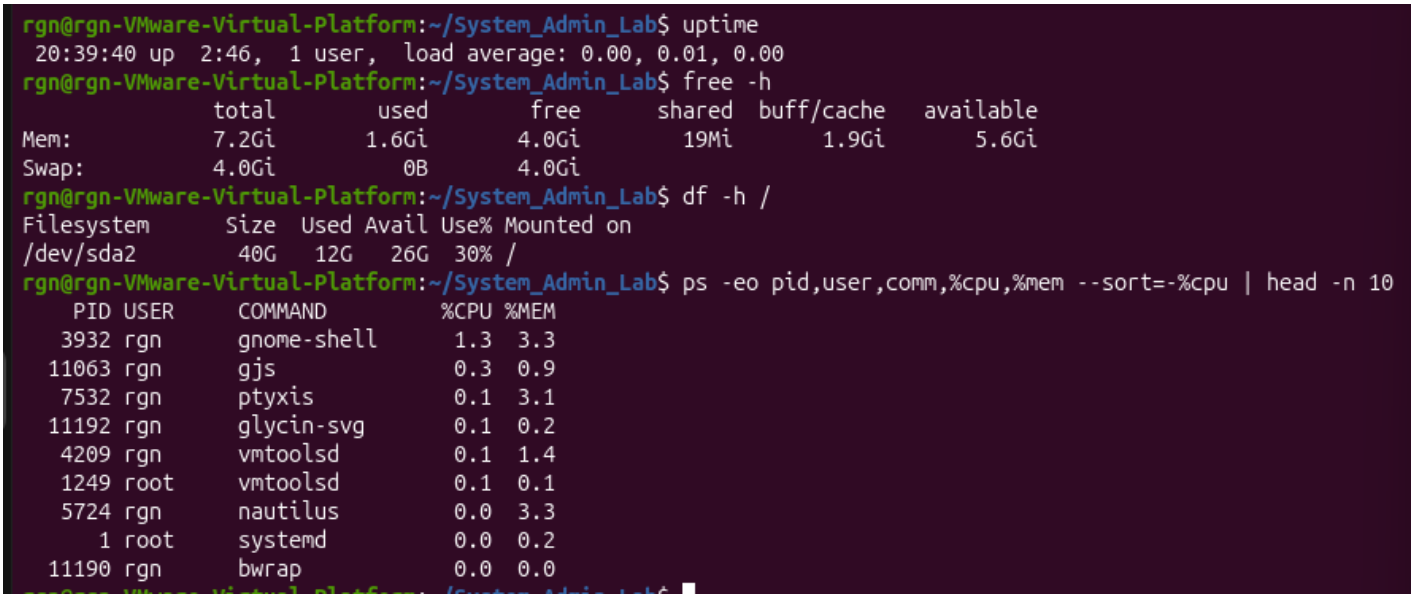


Figure 13. Resource measurements and processes sorted by CPU use.

LECTURE 1 | EXPERIMENT 10

Inspect the network configuration

Objective. Identify active interfaces, the default gateway and listening sockets.

Commands used

```
ip -br addr
ip route
ss -tuln
```

Result. ens33 was up with IPv4 address 192.168.19.131/24. The default gateway was 192.168.19.2. The socket listing showed local DNS listeners and services listening on ports including TCP 22 and TCP 23. This was an inventory of existing services, not a test of their authentication or security.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            UP             192.168.19.131/24 fe80::20c:29ff:fed2:74da/64
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ip route
default via 192.168.19.2 dev ens33 proto dhcp src 192.168.19.131 metric 100
192.168.19.0/24 dev ens33 proto kernel scope link src 192.168.19.131 metric 100
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ss -tuln
Netid     State      Recv-Q     Send-Q                          Local Address:Port          Peer Address:Port
udp       UNCONN     0          0                                     0.0.0.0:43293              0.0.0.0:*
udp       UNCONN     0          0                             192.168.19.131:3702               0.0.0.0:*
udp       UNCONN     0          0                             239.255.255.250:3702              0.0.0.0:*
udp       UNCONN     0          0                                 127.0.0.54:53                 0.0.0.0:*
udp       UNCONN     0          0                               127.0.0.53%lo:53                0.0.0.0:*
udp       UNCONN     0          0                                   127.0.0.1:323                 0.0.0.0:*
udp       UNCONN     0          0                                     0.0.0.0:4500                0.0.0.0:*
udp       UNCONN     0          0                                     0.0.0.0:500                 0.0.0.0:*
udp       UNCONN     0          0                                     0.0.0.0:5353                0.0.0.0:*
udp       UNCONN     0          0                                         *:35482                       *:*
udp       UNCONN     0          0                [fe80::20c:29ff:fed2:74da]%ens33:3702                  [::]:*
udp       UNCONN     0          0                             [ff02::c]%ens33:3702                  [::]:*
udp       UNCONN     0          0                                      [::1]:323                    [::]:*
udp       UNCONN     0          0                                       [::]:4500                   [::]:*
udp       UNCONN     0          0                                       [::]:500                    [::]:*
udp       UNCONN     0          0                                       [::]:5353                   [::]:*
tcp       LISTEN     0          4096                            127.0.0.53%lo:53                0.0.0.0:*
tcp       LISTEN     0          10                                    0.0.0.0:23                  0.0.0.0:*
tcp       LISTEN     0          4096                                  0.0.0.0:22                  0.0.0.0:*
tcp       LISTEN     0          4096                              127.0.0.54:53                   0.0.0.0:*
tcp       LISTEN     0          4096                               127.0.0.1:631                  0.0.0.0:*
tcp       LISTEN     0          4096                                    [::]:22                     [::]:*
tcp       LISTEN     0          4096                                   [::1]:631                    [::]:*
```

Figure 14. Interface addresses, route table and listening TCP and UDP sockets.

LECTURE 1 | EXPERIMENT 11

Test connectivity and name resolution

Objective. Test the local gateway, an external IP address and IPv4 DNS resolution.

Commands used

```
ping -c 4 192.168.19.2
ping -c 4 1.1.1.1
getent ahostsv4 example.com
command -v python3
command -v curl
```

Result. Both ping tests sent four packets and received four replies with 0% packet loss. Name resolution returned IPv4 records for example.com. Python was available at /usr/bin/python3; curl did not yet return a path and was installed in the next experiment.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ping -c 4 192.168.19.2
PING 192.168.19.2 (192.168.19.2) 56(84) bytes of data.
64 bytes from 192.168.19.2: icmp_seq=1 ttl=128 time=0.746 ms
64 bytes from 192.168.19.2: icmp_seq=2 ttl=128 time=4.44 ms
64 bytes from 192.168.19.2: icmp_seq=3 ttl=128 time=0.153 ms
64 bytes from 192.168.19.2: icmp_seq=4 ttl=128 time=0.172 ms

--- 192.168.19.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3103ms
rtt min/avg/max/mdev = 0.153/1.376/4.435/1.781 ms
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ping -c 4 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data.
64 bytes from 1.1.1.1: icmp_seq=1 ttl=128 time=5.47 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=128 time=8.28 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=128 time=4.17 ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=128 time=10.8 ms

--- 1.1.1.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 4.168/7.193/10.848/2.581 ms
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ getent ahostsv4 example.com
172.66.147.243  STREAM example.com
172.66.147.243  DGRAM
172.66.147.243  RAW
104.20.23.154   STREAM
104.20.23.154   DGRAM
104.20.23.154   RAW
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ command -v python3
/usr/bin/python3
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ command -v curl
```

Figure 15. Successful gateway and external connectivity tests and DNS lookup.

LECTURE 1 | EXPERIMENT 12

Install curl and start a local web service

Objective. Install an HTTP testing tool and start a small service bound to the local loopback interface.

Commands used

```
sudo apt install -y curl
curl --version | head -n 2
mkdir -p web_demo
python3 -u -m http.server 8080 --bind 127.0.0.1 --directory web_demo > web_server.log  \
2>&1 &
lab_web_pid=$!
```

Result. APT installed curl successfully, and the version output reported curl 8.18.0. I created web_demo/index.html and started Python HTTP service PID 11868. The process and socket checks confirmed that it was listening on 127.0.0.1:8080.

The package summary reported 20 packages not upgraded. This installation does not establish that all system updates were applied.

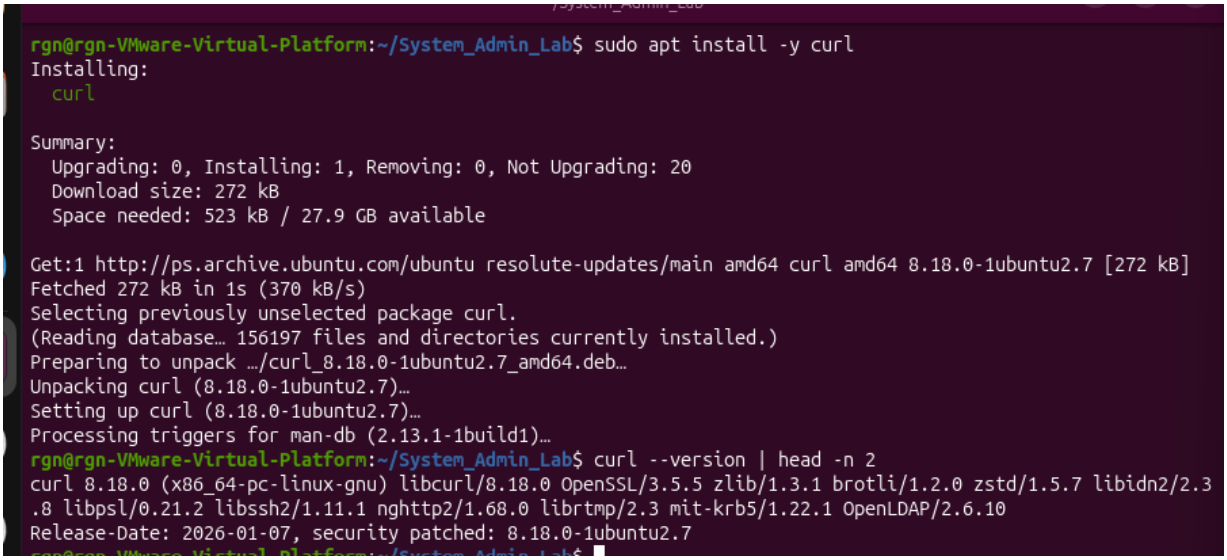


Figure 16. curl package installation and version verification.

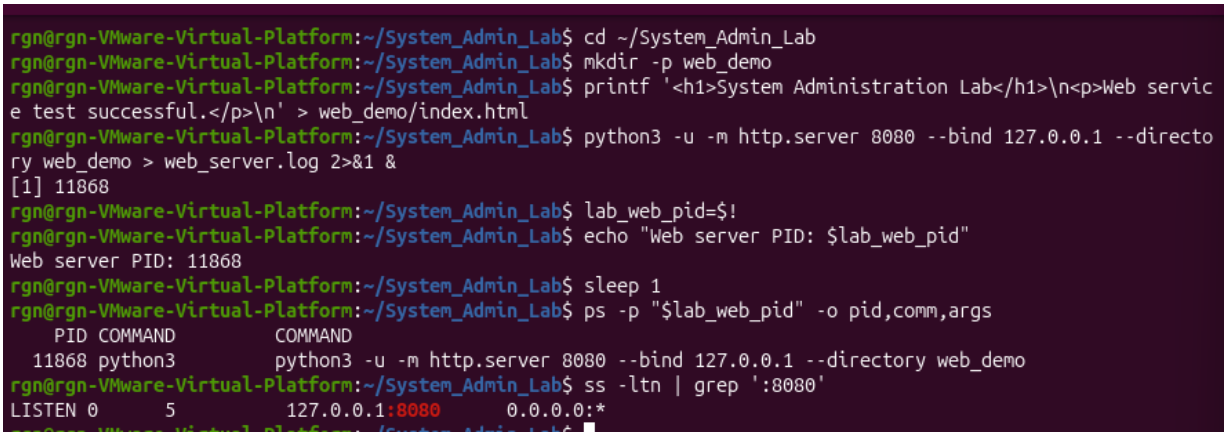


Figure 17. HTML file creation, HTTP process and loopback listening socket.

LECTURE 1 | EXPERIMENT 13

Verify HTTP responses and review web logs

Objective. Check the response headers and body, inspect the request log and stop the owned test process.

Commands used

```
curl -I http://127.0.0.1:8080/
curl http://127.0.0.1:8080/
tail -n 5 web_server.log
kill "$lab_web_pid"
ps -p "$lab_web_pid"
```

Result. The header request returned HTTP/1.0 200 OK and Content-type text/html. The body contained the lab heading and success message. The log recorded HEAD and GET requests with status 200. After stopping the process, the final check printed Lab web server stopped.

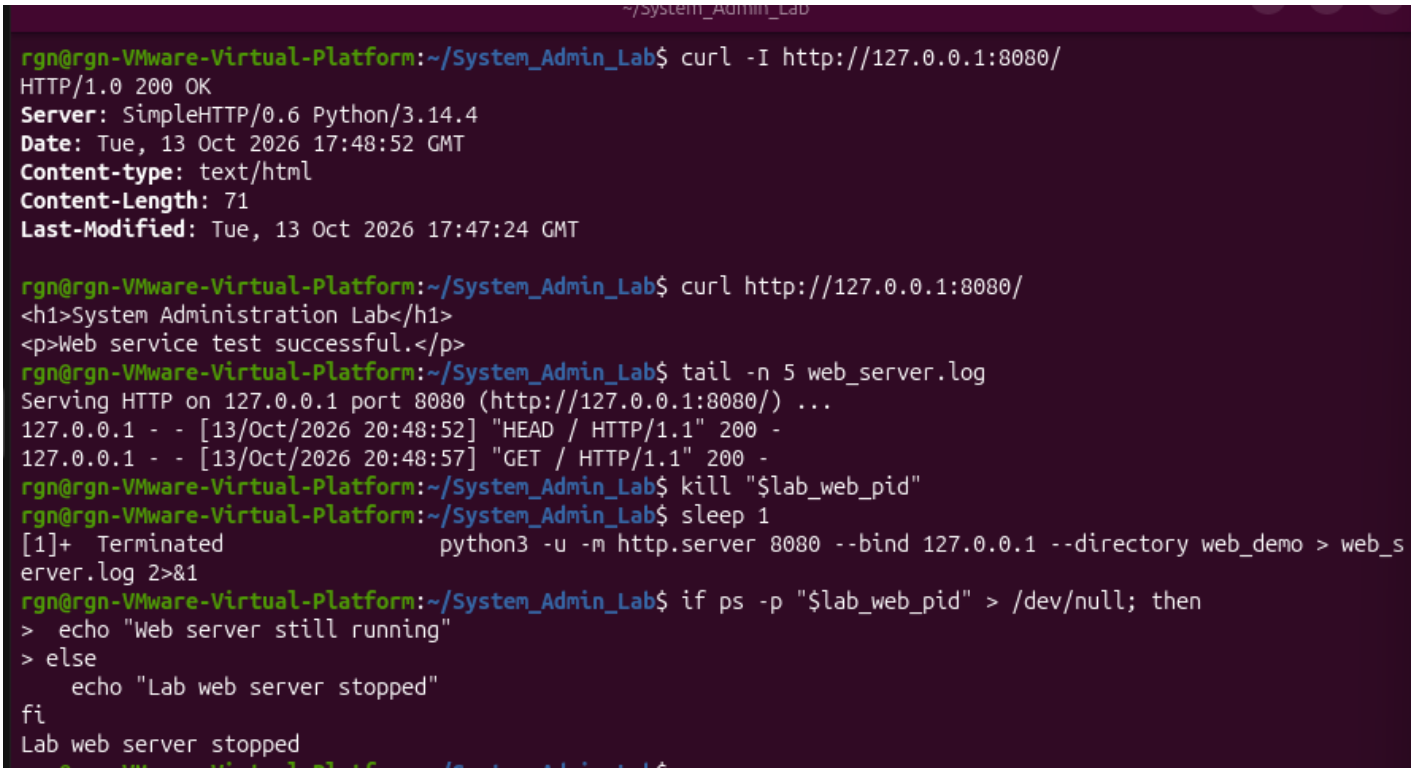


Figure 18. HTTP headers, page contents, request log and confirmed process shutdown.

LECTURE 1 | EXPERIMENT 14

Apply an owner only access policy

Objective. Restrict a lab directory and policy file, then check an attempted read by another user.

Commands used

```
chmod 700 security_demo
chmod 600 security_demo/access_policy.txt
stat -c '%A %a %U:%G %n' security_demo security_demo/access_policy.txt
cat security_demo/access_policy.txt
sudo -u nobody cat "$PWD/security_demo/access_policy.txt"
```

Result. The directory had mode 700 and the file had mode 600, both owned by rgn:rgn. The owner could read the policy text. The attempt by nobody returned Permission denied, showing that the protected path was inaccessible to that user.

The directory permission also restricts traversal. The denied read verifies protection of the path as a whole, rather than isolating only the file permission.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cd ~/System_Admin_Lab
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ mkdir -p security_demo
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "Lab policy: access is restricted to the owner." > s
ecurity_demo/access_policy.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ chmod 700 security_demo
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ chmod 600 security_demo/access_policy.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ stat -c '%A %a %U:%G %n' security_demo security_demo/acce
ss_policy.txt
drwx------ 700 rgn:rgn security_demo
-rw------- 600 rgn:rgn security_demo/access_policy.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat security_demo/access_policy.txt
Lab policy: access is restricted to the owner.
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ sudo -u nobody cat "$PWD/security_demo/access_policy.txt"
cat: /home/rgn/System_Admin_Lab/security_demo/access_policy.txt: Permission denied
```

Figure 19. Permission modes, owner access and denied access by nobody.

LECTURE 1 | EXPERIMENT 15

Document changes and recovery instructions

Objective. Keep a local record of administration changes and a repeatable backup recovery procedure.

Commands used

```
mkdir -p documentation
cat documentation/change_log.txt
cat documentation/restore_procedure.txt
```

Result. The change log records package installation, account work, backups, cron scheduling, monitoring, HTTP checks and the observed inactive UFW status. The recovery procedure lists archive inspection, extraction into a separate directory and checksum comparison.

These are local text records. A wiki server or help-desk ticket system was not deployed. The recovery_check commands are documented instructions; the executed restore used restore/source, as shown in Experiment 4.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat documentation/change_log.txt
System Administration Lab - Change Log
1. Installed and tested ncal and curl.
2. Created, tested and removed sysadmin_lab01.
3. Created a backup script and verified archive restoration.
4. Tested cron backups every minute, then scheduled them daily at 18:00.
5. Checked resources, processes, network, system logs and time synchronization.
6. Tested a local web server, reviewed HTTP logs and stopped it.
7. Firewall inspection result: UFW inactive.
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat documentation/restore_procedure.txt
Backup restoration procedure
1. Work from ~/System_Admin_Lab.
2. List archive contents:
   tar -tzf backups/source_backup.tar.gz
3. Create a separate recovery directory:
   mkdir -p recovery_check
4. Extract the archive:
   tar -xzf backups/source_backup.tar.gz -C recovery_check
5. Compare original and recovered file checksums:
   sha256sum source/notes.txt recovery_check/source/notes.txt
6. Matching checksums confirm identical file contents.
```

Figure 20. Saved change log and written backup restoration procedure.

LECTURE 2 | EXPERIMENT 16

Identify the installed operating system

Objective. Record the guest operating system, kernel, username and working directory.

Commands used

```
whoami
pwd
mkdir -p ~/System_Admin_Lab
cd ~/System_Admin_Lab
cat /etc/os-release
uname -r
```

Result. The session ran as rgn on rgn-VMware-Virtual-Platform. The working directory was /home/rgn/System_Admin_Lab. The OS identified itself as Ubuntu 26.04.1 LTS, and the kernel was 7.0.0-38-generic.

This verifies the state of an installed VM. It is not evidence of the installer screens or of creating the VM.

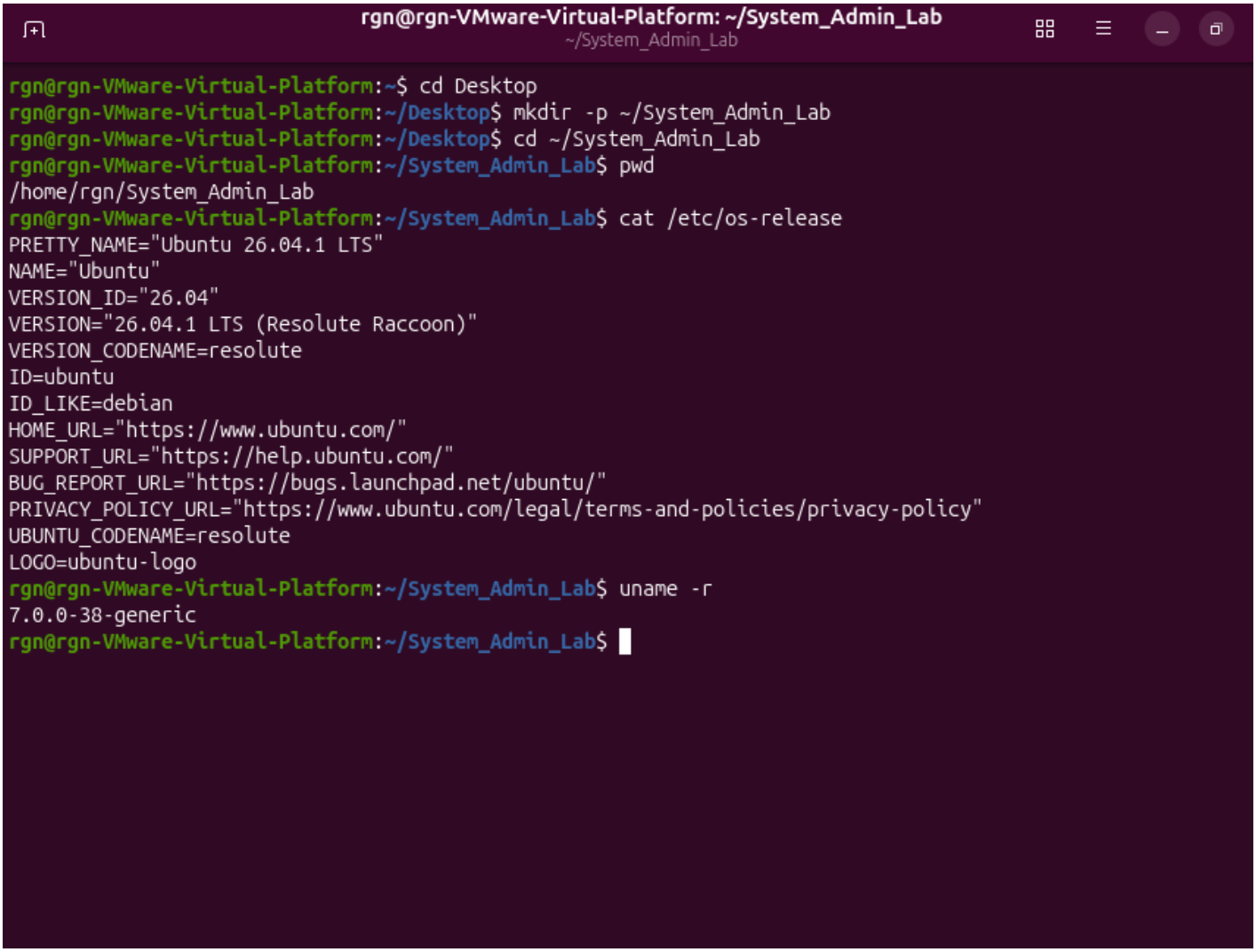


Figure 21. Ubuntu identification, lab directory creation and kernel version.

LECTURE 2 | EXPERIMENT 17

Inspect disks filesystems and free space

Objective. Relate Linux device names to the root filesystem, partitions and installation media.

Commands used

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
df -h /
```

Result. The VM had a 40G disk named sda, including sda1 and an ext4 root partition sda2. The root filesystem had about 12G used and 26G available, with 30% usage. The listing also showed mounted optical media and squashfs loop devices used by snaps.

free -h separately reported 4.0 GiB of swap. Its backing device or file is not identified by these captures. I did not repartition the existing disk.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
NAME     SIZE TYPE FSTYPE   MOUNTPOINTS
fd0        4K disk
loop0      4K loop squashfs /snap/bare/5
loop1   66.8M loop squashfs /snap/core24/1587
loop2  273.7M loop squashfs /snap/firefox/8107
loop3   14.2M loop squashfs /snap/desktop-security-center/206
loop4   66.8M loop squashfs /snap/core24/2124
loop5   19.6M loop squashfs /snap/desktop-security-center/150
loop6   16.5M loop squashfs /snap/firmware-updater/226
loop7    395M loop squashfs /snap/mesa-2404/1165
loop8   91.7M loop squashfs /snap/gtk-common-themes/1535
loop9  606.1M loop squashfs /snap/gnome-46-2404/153
loop10  18.4M loop squashfs /snap/prompting-client/228
loop11  18.8M loop squashfs /snap/prompting-client/204
loop12  15.7M loop squashfs /snap/snap-store/1367
loop13  49.3M loop squashfs /snap/snapd/26865
loop14  44.7M loop squashfs /snap/snapd/28254
loop15   580K loop squashfs /snap/snapd-desktop-integration/361
sda       40G disk
├─sda1     1M part
└─sda2    40G part ext4     /
sr0    109.7M rom  iso9660  /run/media/rgn/CDROM
sr1      6.1G rom  iso9660  /run/media/rgn/Ubuntu 26.04 amd64
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        40G   12G   26G  30% /
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$
```

Figure 22. Disk and partition inventory, filesystem types, mount points and root space.

LECTURE 2 | EXPERIMENT 18

Check services logs firewall and time

Objective. Review installed-system health, recent warning messages, firewall state and time synchronisation.

Commands used

```
systemctl --failed --no-pager
sudo journalctl -b -p warning -n 10 --no-pager
sudo ufw status verbose
timedatectl status
```

Result. The system-level failed-unit query returned 0 loaded units. The journal nevertheless contained session warnings and user-service errors, including snap desktop integration. UFW reported inactive. The guest time zone was Asia/Hebron, the system clock was synchronised and the NTP service was active.

The scope of the failed-unit query is system units; its result does not mean that every user service or logged event was error-free. Firewall inspection was completed, but firewall enabling and rule testing were not recorded.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ systemctl --failed --no-pager
  UNIT LOAD ACTIVE SUB DESCRIPTION

0 loaded units listed.
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ sudo journalctl -b -p warning -n 10 --no-pager
Oct 13 20:30:40 rgn-VMware-Virtual-Platform systemd-logind[1636]: Existing logind session ID 3 used by new au
dit session, ignoring.
Oct 13 20:30:40 rgn-VMware-Virtual-Platform systemd[10442]: /usr/lib/systemd/user/spice-vdagent.service:23: U
nknown key 'StandardError' in section [Install], ignoring.
Oct 13 20:30:40 rgn-VMware-Virtual-Platform su[10432]: pam_systemd(su-l:session): Failed to check if /run/use
r/1001/bus exists, ignoring: Permission denied
Oct 13 20:30:40 rgn-VMware-Virtual-Platform systemd[10442]: ubuntu-insights-collect.service: unit configures
an IP firewall, but not running as root.
Oct 13 20:30:40 rgn-VMware-Virtual-Platform systemd[10442]: ubuntu-insights-collect.service: (This warning is
 only shown for the first unit using IP firewalling.)
Oct 13 20:31:02 rgn-VMware-Virtual-Platform systemd[10442]: xdg-document-portal.service: Failed to kill contr
ol group /user.slice/user-1001.slice/user@1001.service/session.slice/xdg-document-portal.service, ignoring: O
peration not permitted
Oct 13 20:31:02 rgn-VMware-Virtual-Platform systemd[10442]: snap.snapd-desktop-integration.snapd-desktop-inte
gration.service: Failed with result 'exit-code'.
Oct 13 20:37:19 rgn-VMware-Virtual-Platform update-notifier[4171]: gtk_widget_get_scale_factor: assertion 'GT
K_IS_WIDGET (widget)' failed
Oct 13 20:37:19 rgn-VMware-Virtual-Platform update-notifier[4171]: gtk_widget_get_scale_factor: assertion 'GT
K_IS_WIDGET (widget)' failed
Oct 13 20:37:19 rgn-VMware-Virtual-Platform update-notifier[4171]: gtk_widget_get_scale_factor: assertion 'GT
K_IS_WIDGET (widget)' failed
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ sudo ufw status verbose
Status: inactive
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ timedatectl status
               Local time: Tue 2026-10-13 20:44:26 EEST
           Universal time: Tue 2026-10-13 17:44:26 UTC
                 RTC time: Tue 2026-10-13 17:44:26
                Time zone: Asia/Hebron (EEST, +0300)
System clock synchronized: yes
              NTP service: active
          RTC in local TZ: no
```

Figure 23. System failed-unit check, genuine journal warnings, firewall state and time status.

LECTURE 3 | EXPERIMENT 19

Create and list files and directories

Objective. Create a small working tree, include a hidden file and verify the contents through CLI and GUI views.

Commands used

```
mkdir -p source copies archive
printf 'System administration lab\nLinux commands practice\n' > source/notes.txt
touch source/empty.txt source/.hidden.txt
ls -la source
cat source/notes.txt
```

Result. The source directory contained a 50-byte notes.txt, an empty file and .hidden.txt. ls -la displayed the hidden file, while the GUI view showed the ordinary visible files. The displayed text matched the content written to notes.txt.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cd ~/System_Admin_Lab
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ mkdir -p source copies archive
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ printf 'System administration lab\nLinux commands practic
e\n' > source/notes.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ touch source/empty.txt source/.hidden.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ls -la source
total 12
drwxrwxr-x 2 rgn rgn 4096 Oct 13 18:01 .
drwxrwxr-x 5 rgn rgn 4096 Oct 13 18:01 ..
-rw-rw-r-- 1 rgn rgn    0 Oct 13 18:01 .hidden.txt
-rw-rw-r-- 1 rgn rgn    0 Oct 13 18:01 empty.txt
-rw-rw-r-- 1 rgn rgn   50 Oct 13 18:01 notes.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat source/notes.txt
System administration lab
Linux commands practice
```

Figure 24. File creation, detailed listing and text verification.



Figure 25. GUI view of the three lab directories.



Figure 26. GUI view of notes.txt and empty.txt.

LECTURE 3 | EXPERIMENT 20

Copy move and rename files

Objective. Copy a file, preserve a directory copy and rename the copied file.

Commands used

```
cp source/notes.txt copies/notes_copy.txt
cp -rp source archive/source_backup
mv copies/notes_copy.txt copies/renamed_notes.txt
ls -l copies
ls -la archive/source_backup
cat copies/renamed_notes.txt
```

Result. The copied file was renamed to renamed_notes.txt. The directory copy contained the hidden and ordinary source files. Reading the renamed copy showed the same two text lines as the original.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cp source/notes.txt copies/notes_copy.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cp -rp source archive/source_backup
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ mv copies/notes_copy.txt copies/renamed_notes.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ls -l copies
total 4
-rw-rw-r-- 1 rgn rgn 50 Oct 13 18:03 renamed_notes.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ls -la archive/source_backup
total 12
drwxrwxr-x 2 rgn rgn 4096 Oct 13 18:01 .
drwxrwxr-x 3 rgn rgn 4096 Oct 13 18:03 ..
-rw-rw-r-- 1 rgn rgn    0 Oct 13 18:01 .hidden.txt
-rw-rw-r-- 1 rgn rgn    0 Oct 13 18:01 empty.txt
-rw-rw-r-- 1 rgn rgn   50 Oct 13 18:01 notes.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat copies/renamed_notes.txt
System administration lab
Linux commands practice
```

Figure 27. Copy operations, renamed file and copied directory listing.

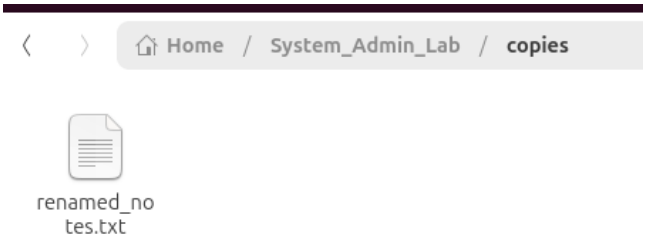


Figure 28. GUI confirmation of renamed_notes.txt.

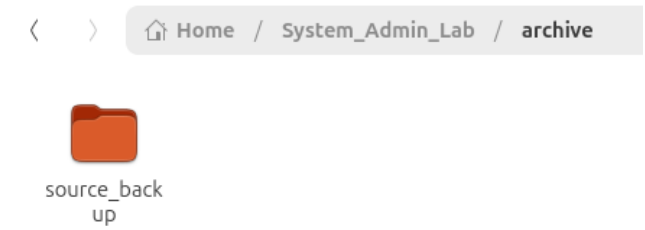


Figure 29. GUI confirmation of the source_backup directory.

LECTURE 3 | EXPERIMENT 21

Navigate paths and remove test files

Objective. Use absolute and relative paths, then remove disposable files and directories.

Commands used

```
cd source
pwd
cd ..
cd /home/rgn/System_Admin_Lab/archive/source_backup
cd ~
rm deletion_demo/full_dir/delete_me.txt
rmdir deletion_demo/empty_dir deletion_demo/renamed_dir
rm -r deletion_demo/full_dir
```

Result. pwd confirmed the selected directory after relative and absolute navigation. In the deletion exercise, a file was moved, a directory was renamed, and the temporary files and directories were removed. The final deletion_demo listing contained only . and .. entries.

An initial standalone v typo produced command not found; the intended mv command was then executed. Removal commands were confined to the lab deletion_demo tree.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cd ~/System_Admin_Lab
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ pwd
/home/rgn/System_Admin_Lab
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cd source
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab/source$ pwd
/home/rgn/System_Admin_Lab/source
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab/source$ cd ..
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ pwd
/home/rgn/System_Admin_Lab
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cd /home/rgn/System_Admin_Lab/archive/source_backup
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab/archive/source_backup$ pwd
/home/rgn/System_Admin_Lab/archive/source_backup
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab/archive/source_backup$ cd ~
rgn@rgn-VMware-Virtual-Platform:~$ pwd
/home/rgn
rgn@rgn-VMware-Virtual-Platform:~$ cd ~/System_Admin_Lab
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$
```

Figure 30. Relative paths, absolute paths and home-directory navigation.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ mkdir -p deletion_demo/empty_dir deletion_demo/full_dir
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ touch deletion_demo/delete_me.txt deletion_demo/full_dir/
sample.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ v
v: command not found
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ mv deletion_demo/delete_me.txt deletion_demo/full_dir/
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ mv deletion_demo/empty_dir deletion_demo/renamed_dir
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ls -R deletion_demo
deletion_demo:
full_dir  renamed_dir

deletion_demo/full_dir:
delete_me.txt  sample.txt

deletion_demo/renamed_dir:
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ rm deletion_demo/full_dir/delete_me.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ rmdir deletion_demo/renamed_dir
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ rm -r deletion_demo/full_dir
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ls -la deletion_demo
total 8
drwxrwxr-x 2 rgn rgn 4096 Oct 13 18:16 .
drwxrwxr-x 6 rgn rgn 4096 Oct 13 18:16 ..
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$
```

Figure 31. Move, rename and removal operations in the disposable deletion tree.

LECTURE 3 | EXPERIMENT 22

Inspect and sort structured text

Objective. Read selected lines, count the file and sort by text or a numeric field.

Commands used

```
head -n 2 records.txt
tail -n 2 records.txt
wc -l records.txt
wc -w records.txt
wc -c records.txt
sort records.txt
sort -r records.txt
sort -t: -n -k3,3 records.txt
```

Result. records.txt contained four colon-separated records. The counts were 4 lines, 4 words and 60 bytes. Colons do not split words for wc -w. Numeric sorting by field 3 placed the marks in order 10, 20, 20 and 30.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cd ~/System_Admin_Lab
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ printf 'Sara:student:30\nAli:student:20\nOmar:admin:10\nA
li:student:20\n' > records.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat records.txt
Sara:student:30
Ali:student:20
Omar:admin:10
Ali:student:20
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ head -n 2 records.txt
Sara:student:30
Ali:student:20
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ tail -n 2 records.txt
Omar:admin:10
Ali:student:20
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ wc -l records.txt
4 records.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ wc -w records.txt
4 records.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ wc -c records.txt
60 records.txt
```

Figure 32. Source records, head and tail results, and line word and byte counts.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ sort records.txt
Ali:student:20
Ali:student:20
Omar:admin:10
Sara:student:30
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ sort -r records.txt
Sara:student:30
Omar:admin:10
Ali:student:20
Ali:student:20
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ sort -t: -n -k3,3 records.txt
Omar:admin:10
Ali:student:20
Ali:student:20
Sara:student:30
```

Figure 33. Normal, reverse and numeric field sorting.

LECTURE 3 | EXPERIMENT 23

Filter merge and transform text

Objective. Remove adjacent duplicates, extract fields, search records, merge files and translate characters.

Commands used

```
uniq records.txt
sort records.txt | uniq
cut -d: -f1,3 records.txt
grep student records.txt
paste names.txt marks.txt
tr 'a-z' 'A-Z' < names.txt
tr ':' ' ' < records.txt
```

Result. uniq alone retained the non-adjacent duplicate Ali record. Sorting before uniq removed it. cut selected the name and mark fields; grep returned the three student records. paste paired names with marks, and tr produced uppercase names and space-separated records.

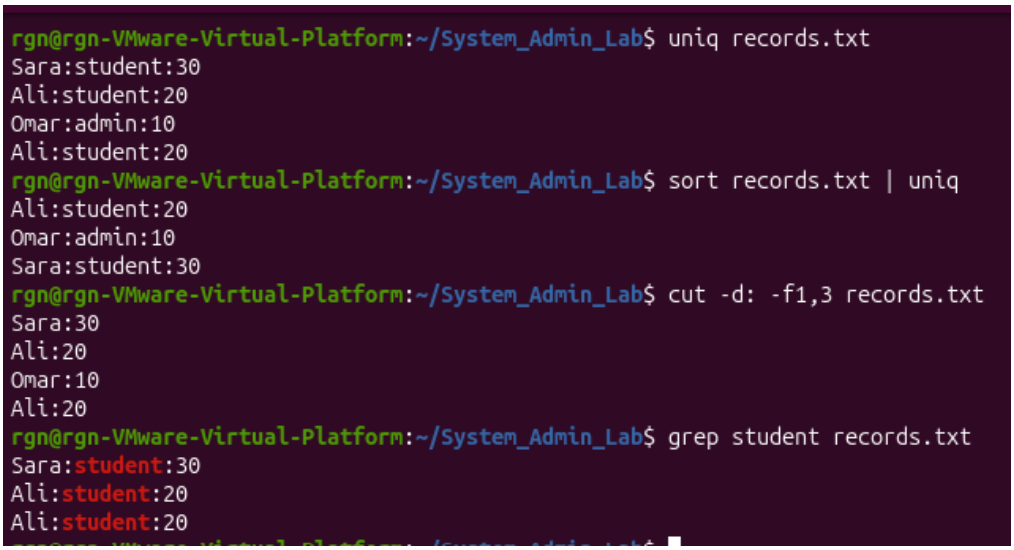


Figure 34. Duplicate handling, selected fields and matching student records.

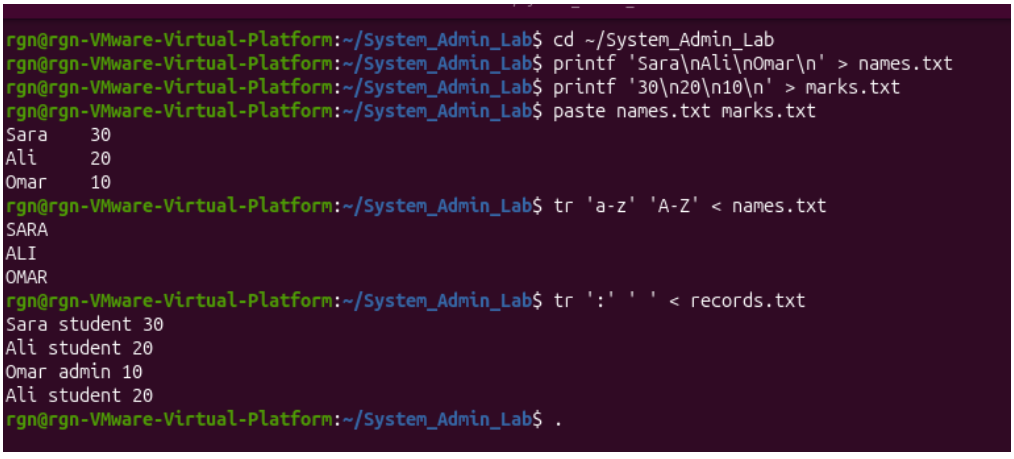


Figure 35. Merged name and mark columns and character transformations.

LECTURE 3 | EXPERIMENT 24

Read a command manual

Objective. Open the ls manual and inspect its command description and options.

Commands used

```
man ls
```

Result. The manual page displayed the name, synopsis, description and command options for ls. This provides a direct source of command syntax and option meanings during administration.

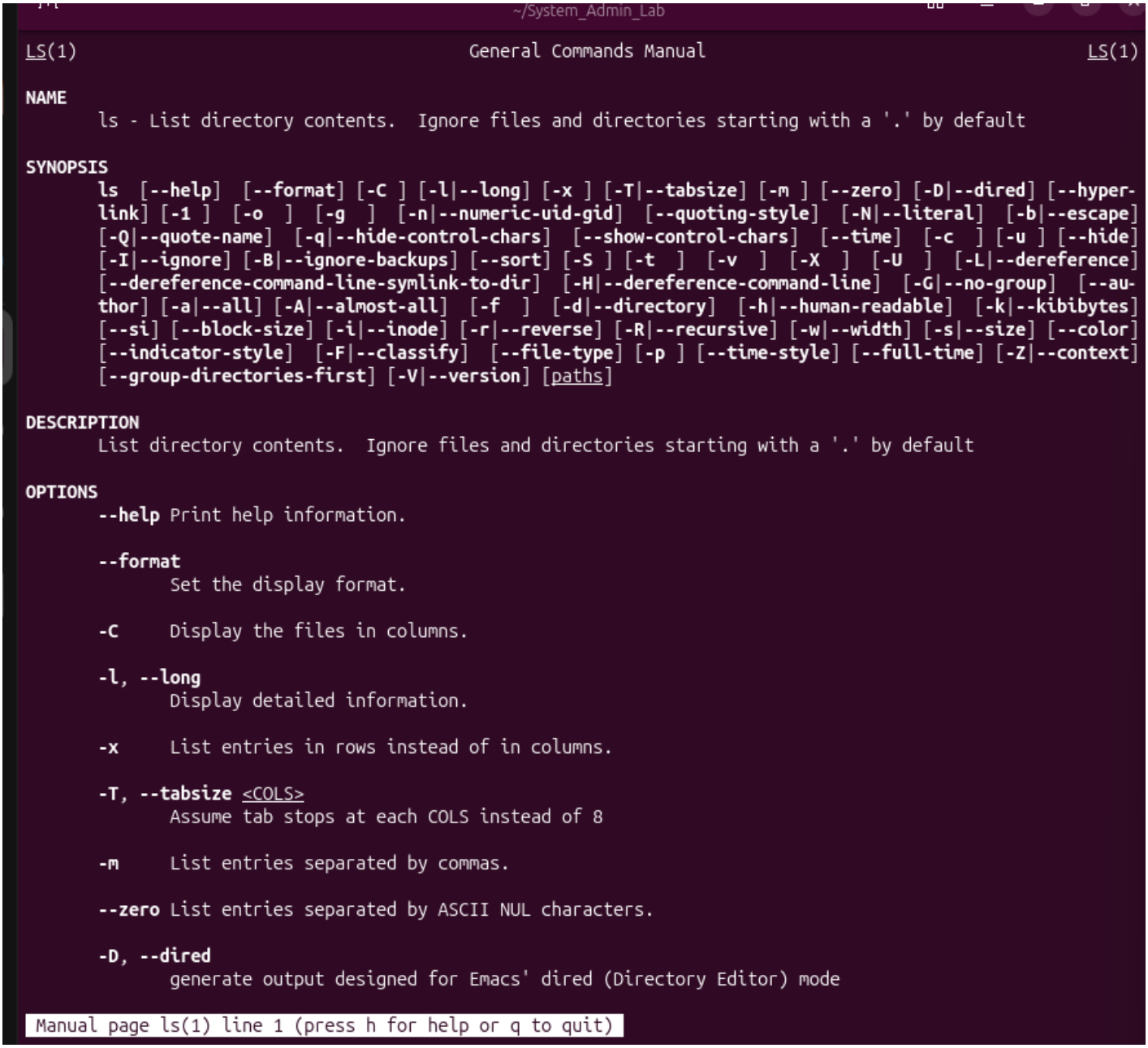


Figure 36. The actual ls manual page displayed in the terminal.

LECTURE 3 | EXPERIMENT 25

Search manuals and inspect the hierarchy

Objective. Find manual entries by keyword, read a section introduction and inspect standard system directories.

Commands used

```
man -k directory | head -n 5
man 1 intro | head -n 8
ls -ld /bin /sbin /usr/bin /usr/sbin /etc /home /var /tmp
```

Result. The keyword search returned directory-related entries. Section 1 intro described user commands. The directory listing showed the standard paths; on this Ubuntu system, /bin and /sbin appeared as links into /usr.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ man -k directory | head -n 5
alphasort (3)        - scan a directory for matching entries
basename (1)         - Print NAME with any leading directory components removed If specified, also remove ...
bindtextdomain (3)   - set directory containing message catalogs
chacl (1)            - change the access control list of a file or directory
chdir (2)            - change working directory
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ man 1 intro | head -n 8
intro(1)                                General Commands Manual                                intro(1)

NAME
       intro - introduction to user commands

DESCRIPTION
       Section  1  of  the  manual  describes user commands and tools, for example, file manipulation tools,
       shells, compilers, web browsers, file and image viewers and editors, and so on.
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ls -ld /bin /sbin /usr/bin /usr/sbin /etc /home /var /tmp
lrwxrwxrwx   1 root root     7 Apr 20 11:46 /bin -> usr/bin
drwxr-xr-x 145 root root 12288 Oct 13 12:28 /etc
drwxr-xr-x   3 root root  4096 Oct  8 15:27 /home
lrwxrwxrwx   1 root root     8 Apr 20 11:46 /sbin -> usr/sbin
drwxrwxrwt  18 root root   400 Oct 13 18:52 /tmp
drwxr-xr-x   2 root root 49152 Oct 13 18:41 /usr/bin
drwxr-xr-x   2 root root 20480 Oct 12 18:24 /usr/sbin
drwxr-xr-x  14 root root  4096 Oct  4 17:31 /var
```

Figure 37. Manual keyword search, section introduction and filesystem hierarchy inspection.

LECTURE 3 | EXPERIMENT 26

View a long file with more

Objective. Create a longer text file and display it using a pager.

Commands used

```
seq 1 100 > long.txt
more long.txt
```

Result. The screenshot shows the beginning of the generated number sequence in the more session. It demonstrates viewing a file through the pager; the complete 100-line file is not reproduced in the screenshot.

A separate less launch-only capture was excluded because it did not document the interactive pager output.

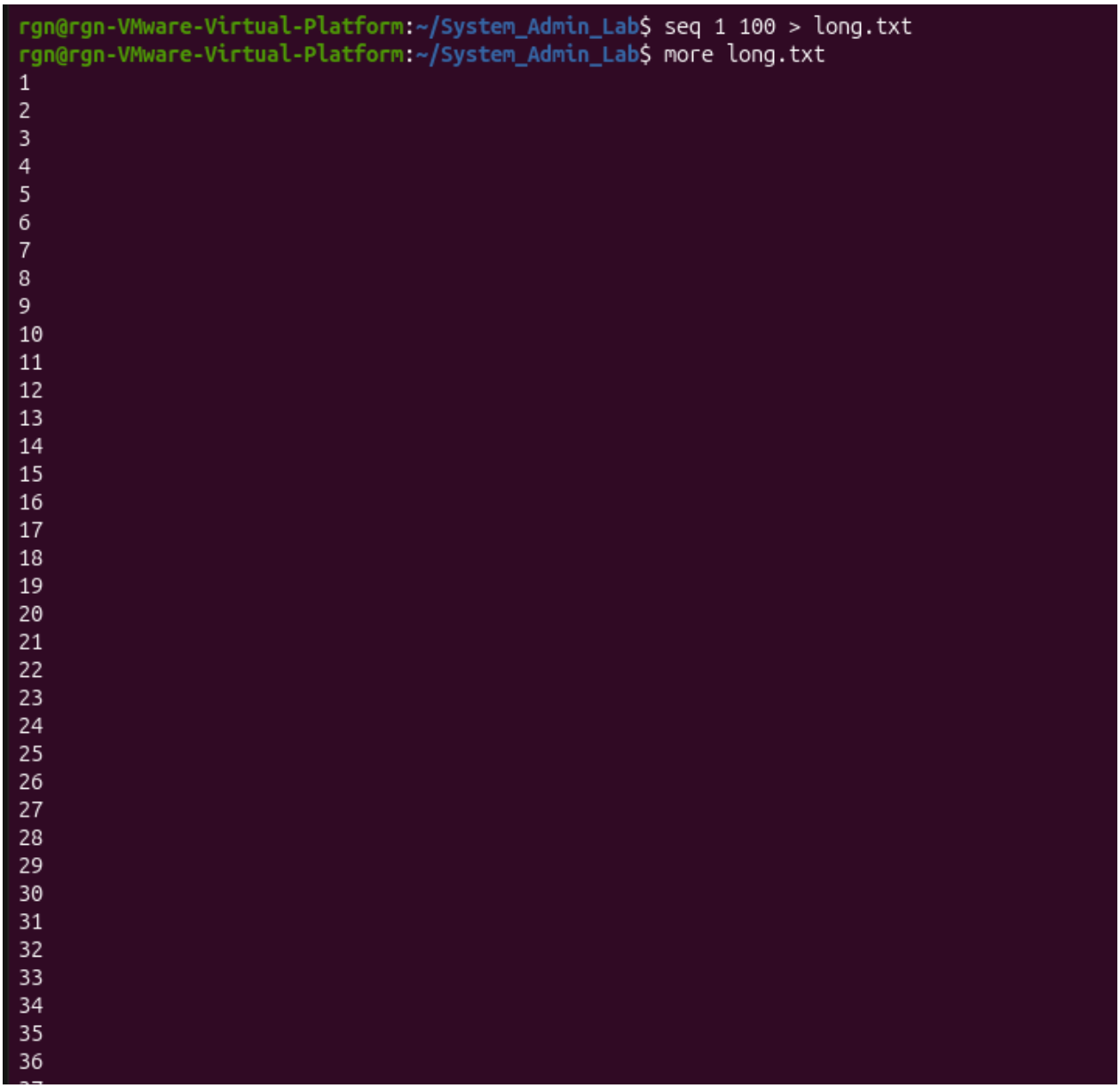


Figure 38. Number sequence displayed after launching more.

LECTURE 3 | EXPERIMENT 27

Use date echo and the calendar

Objective. Display time, format a date, print a string and install the calendar utility.

Commands used

```
date
date +'%Y-%m-%d %H:%M:%S'
echo "System administration lab"
sudo apt install -y ncal
dpkg -s ncal | head -n 5
cal
```

Result. date and echo produced the expected displayed values. ncal was installed, its package status read install ok installed, and cal displayed October 2026.

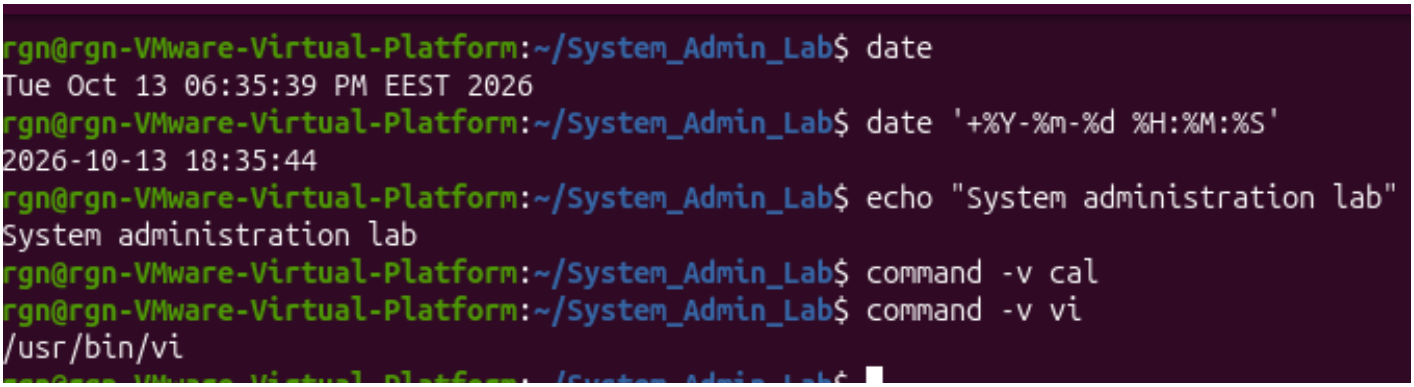


Figure 39. Date output, formatted timestamp and echo output.

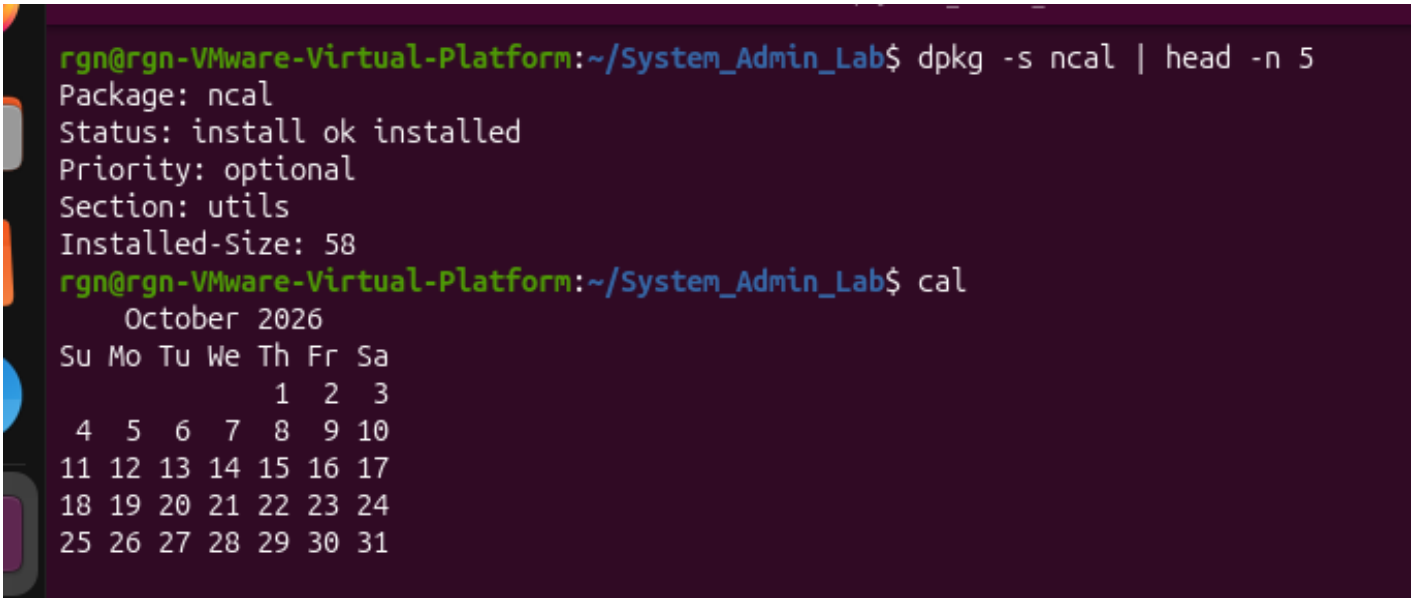


Figure 40. Installed calendar package and the October 2026 calendar.

LECTURE 3 | EXPERIMENT 28

Edit and save a file with vi

Objective. Enter text in vi, save the file and verify the saved contents from the shell.

Commands used

```
vi vi_practice.txt
# Editor sequence: i, enter text, Esc, :wq, Enter
cat vi_practice.txt
```

Result. The editor capture contains three lab text lines. The subsequent cat output shows the same lines in vi_practice.txt, confirming that the text was saved. The exercise used insert mode for typing, command mode after Esc, and an ex command to write and quit.

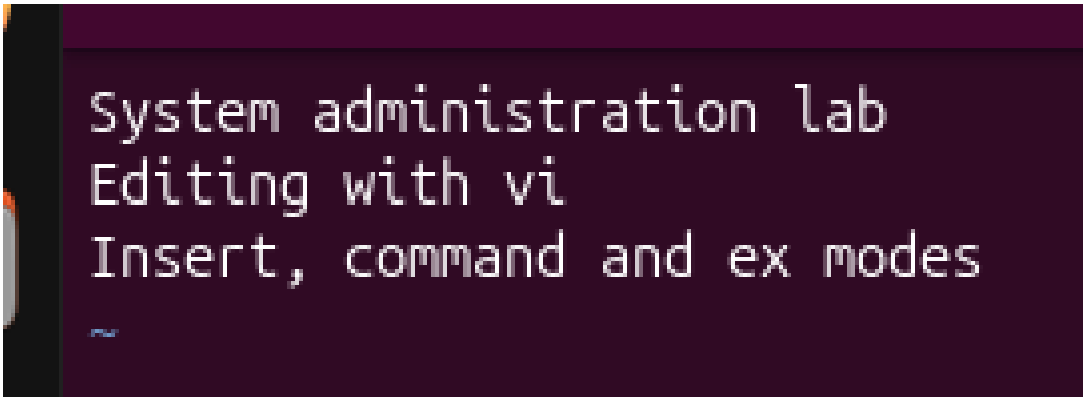


Figure 41. Lab text inside the vi editor.

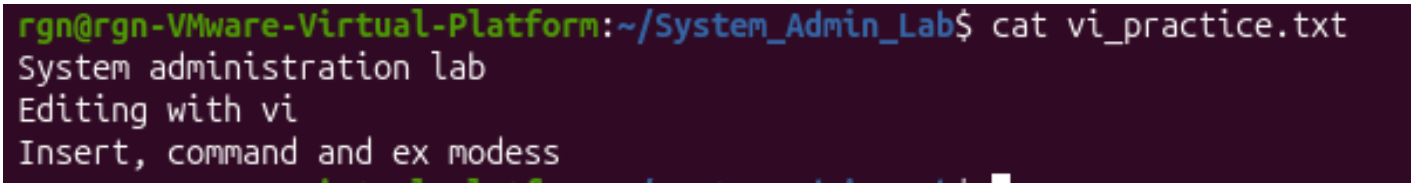


Figure 42. Saved file contents verified with cat.

LECTURE 4 | EXPERIMENT 29

Identify Bash and use command history

Objective. Record the configured shell and running Bash version, then repeat commands from history.

Commands used

```
echo "$SHELL"
bash --version | head -n 1
echo "$BASH_VERSION"
tty
cat /etc/shells
history | tail -n 8
!!
```

Result. The configured shell was /bin/bash and Bash reported version 5.3.9. tty identified /dev/pts/0. The history list contained the test echo commands, and !! repeated history-demo-two.

$SHELL records the configured login shell; by itself it is not a reliable test of which shell is currently executing. The Bash version variables and version command identify Bash here.

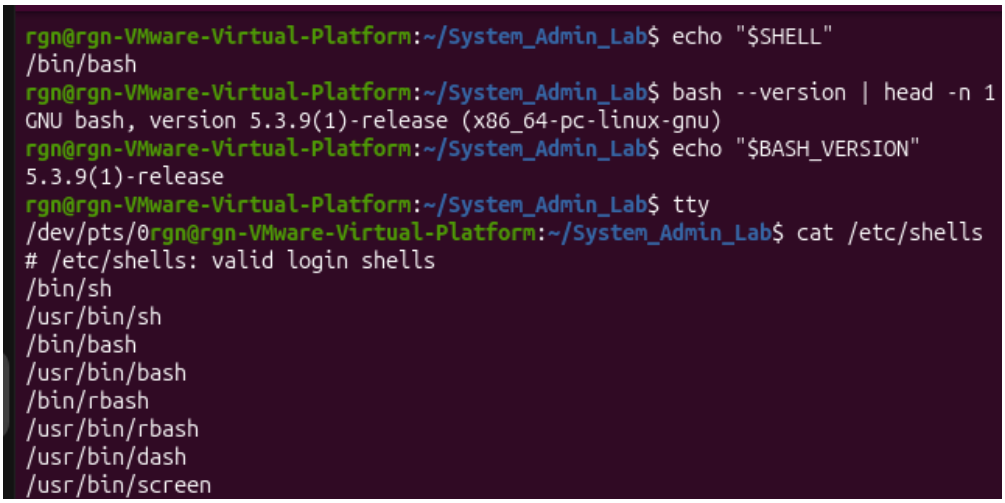


Figure 43. Shell setting, Bash version, terminal device and available shell paths.

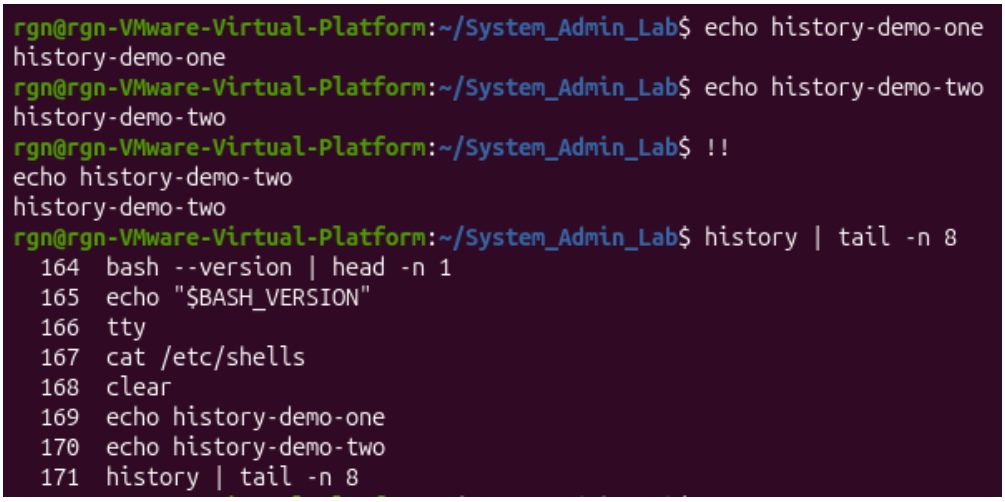


Figure 44. History listing and repetition of the previous command.

LECTURE 4 | EXPERIMENT 30

Use aliases and case sensitive variables

Objective. Recall a numbered history entry, define an alias and compare variables with different letter case.

Commands used

```
!169
alias labls='ls -la'
alias labls
labls source
unalias labls
lab_name="Ahmad"
lab_Name="Nassar"
echo "${lab_name} ${lab_Name}"
```

Result. !169 repeated history-demo-one. The labls alias expanded to a detailed listing and was then removed. lab_name and lab_Name held separate values, demonstrating case-sensitive variable names; the combined echo displayed Ahmad Nassar.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ !169
echo history-demo-one
history-demo-one
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cd ~/System_Admin_Lab
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ alias labls='ls -la'
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ alias labls
alias labls='ls -la'
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ labls source
total 12
drwxrwxr-x 2 rgn rgn 4096 Oct 13 18:01 .
drwxrwxr-x 6 rgn rgn 4096 Oct 13 18:51 ..
-rw-rw-r-- 1 rgn rgn    0 Oct 13 18:01 .hidden.txt
-rw-rw-r-- 1 rgn rgn    0 Oct 13 18:01 empty.txt
-rw-rw-r-- 1 rgn rgn   50 Oct 13 18:01 notes.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ unalias labls
```

Figure 45. Numbered history expansion and alias definition use and removal.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ lab_name="Ahmad"
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ lab_Name="Nassar"
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "$lab_name"
Ahmad
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "$lab_Name"
Nassar
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "${lab_name} ${lab_Name}"
Ahmad Nassar
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ set | grep -E '^(lab_name|lab_Name)='
lab_Name=Nassar
lab_name=Ahmad
```

Figure 46. Two distinct variable names and their displayed values.

LECTURE 4 | EXPERIMENT 31

Export variables and inspect the environment

Objective. Compare child-shell access before and after export, then read common shell settings.

Commands used

```
lab_export_demo="Linux Shells"
bash -c 'echo "Before export: [$lab_export_demo]"'
export lab_export_demo
printenv lab_export_demo
bash -c 'echo "After export: [$lab_export_demo]"'
unset lab_export_demo
echo "$HOME" "$SHELL" "$UID" "$PATH" "$PS1" "$PS2"
```

Result. The child shell showed an empty value before export and Linux Shells after export. printenv also returned the exported value. The separate settings capture showed HOME=/home/rgn, SHELL=/bin/bash, UID=1000, the command search path and prompt settings.

Ordinary shell variables can be assigned without export. export makes a variable available in the environment inherited by child processes.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ lab_export_demo="Linux Shells"
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ bash -c 'echo "Before export: [$lab_export_demo]"'
Before export: []
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ export lab_export_demo
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ printenv lab_export_demo
Linux Shells
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ bash -c 'echo "After export: [$lab_export_demo]"'
After export: [Linux Shells]
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ unset lab_export_demo
```

Figure 47. Local variable, exported environment variable and child-shell comparison.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "HOME=$HOME"
HOME=/home/rgn
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "SHELL=$SHELL"
SHELL=/bin/bash
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "UID=$UID"
UID=1000
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "PATH=$PATH"
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap
/bin
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ printf 'PS1=%s\nPS2=%s\n' "$PS1" "$PS2"
PS1=\[\e]0;\u@\h: \w\a\]${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;34m\]\w\
[\033[00m\]\$
PS2=>
```

Figure 48. Home, shell, user ID, search path and prompt settings.

LECTURE 4 | EXPERIMENT 32

Inspect startup files and source a configuration

Objective. Locate Bash startup files and load a separate lab configuration into the current shell.

Commands used

```
ls -l /etc/profile /etc/bash.bashrc ~/.bashrc ~/.profile
ls /etc/profile.d
source ./labrc
echo "$lab_course"
labpwd
unalias labpwd
unset lab_course
```

Result. The listing confirmed the Ubuntu startup-file locations. labrc defined lab_course and the labpwd alias. source applied them to the current shell, echo printed System Administration and labpwd printed the lab directory. The alias and variable were then removed.

The exercise sourced a separate lab file. It did not modify the system-wide or personal startup files.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cd ~/System_Admin_Lab
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ls -l /etc/profile /etc/bash.bashrc ~/.bashrc ~/.profile
-rw-r--r-- 1 root root 2553 Feb 13  2026 /etc/bash.bashrc
-rw-r--r-- 1 root root  641 Apr 20 11:46 /etc/profile
-rw-r--r-- 1 rgn  rgn  3771 Feb 13  2026 /home/rgn/.bashrc
-rw-r--r-- 1 rgn  rgn   807 Feb 13  2026 /home/rgn/.profile
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ls /etc/profile.d
01-locale-fix.sh           Z99-cloudinit-warnings.sh  debuginfod.csh            vte-2.91.sh
70-systemd-shell-extra.sh  apps-bin-path.sh           debuginfod.sh             vte.csh
80-systemd-osc-context.sh  bash_completion.sh         gnome-session_gnomerc.sh  xdg_dirs_desktop_session.sh
Z99-cloud-locale-test.sh   cedilla-portuguese.sh      im-config_wayland.sh
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ printf 'lab_course="System Administration"\nalias labpwd=
"pwd"\n' > labrc
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat labrc
lab_course="System Administration"
alias labpwd="pwd"
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ source ./labrc
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "$lab_course"
System Administration
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ labpwd
/home/rgn/System_Admin_Lab
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ unalias labpwd
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ unset lab_course
```

Figure 49. Startup-file inventory, labrc creation, sourcing and verification.

LECTURE 4 | EXPERIMENT 33

Use logical operators background jobs and pipes

Objective. Control execution by exit status, observe a background job and pass one command output into another.

Commands used

```
echo "First"; echo "Second"
true && echo "AND: second command ran"
false || echo "OR: recovery command ran"
touch logic_first.txt && touch logic_second.txt
sleep 20 &
lab_bg_pid=$!
jobs
wait "$lab_bg_pid"
grep student records.txt | wc -l
```

Result. The semicolon ran both echo commands. && ran the second command after success; || ran the recovery command after failure. The complementary cases were skipped as expected. The background job was shown running and later completed. The pipeline counted three matching student records.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "First"; echo "Second"
First
Second
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ true && echo "AND: second command ran"
AND: second command ran
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ false && echo "AND: skipped"
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ false || echo "OR: recovery command ran"
OR: recovery command ran
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ true || echo "OR: skipped"
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ touch logic_first.txt && touch logic_second.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ls -l logic_first.txt logic_second.txt
-rw-rw-r-- 1 rgn rgn 0 Oct 13 19:10 logic_first.txt
-rw-rw-r-- 1 rgn rgn 0 Oct 13 19:10 logic_second.txt
```

Figure 50. Sequential and conditional execution and the created logic files.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ sleep 20 &
[1] 8169
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ lab_bg_pid=$!
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "Background PID: $lab_bg_pid"
Background PID: 8169
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ jobs -l
[1]+  8169 Running                 sleep 20 &
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ wait "$lab_bg_pid"
[1]+  Done                    sleep 20
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "Background task finished"
Background task finished
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ grep student records.txt | wc -l
3
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ps -ef | grep '[b]ash' | head -n 5
rgn         7583    7548  0 18:49 pts/0    00:00:00 /usr/bin/bash
```

Figure 51. Background job, wait completion and pipeline count.

LECTURE 4 | EXPERIMENT 34

Expand filename patterns

Objective. Compare star, question mark, bracket ranges and negated bracket patterns.

Commands used

```
ls glob_demo/*.txt
ls glob_demo/file?.txt
ls glob_demo/file[1-2].txt
ls glob_demo/file[^1-2].txt
```

Result. The star pattern matched all .txt files. file?.txt matched file1, file2 and fileA, but excluded file10 because ? matches one character. The range selected file1 and file2; the negated range selected fileA.

The trailing backslash in the creation command continued the command on the next physical line. Globbing expanded existing pathnames before ls ran.

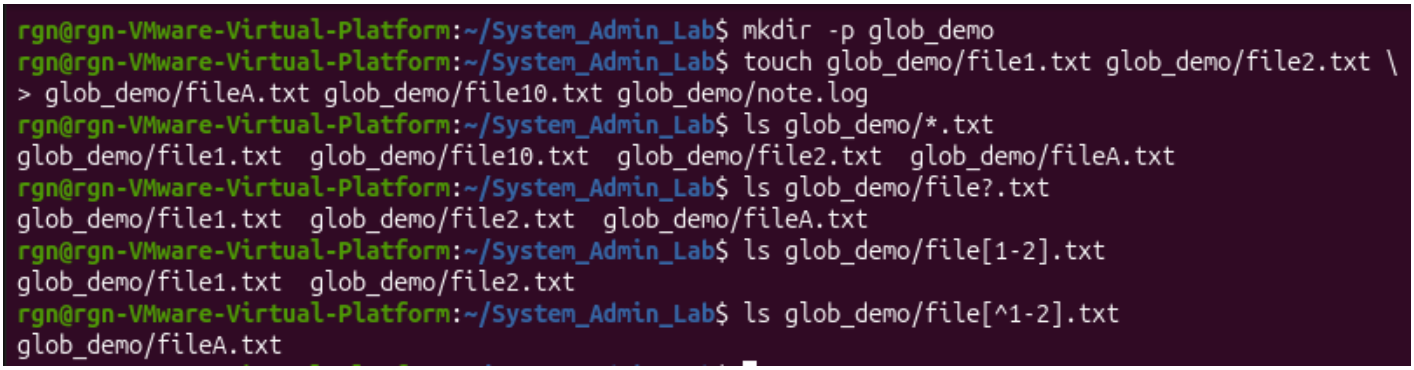


Figure 52. Filename creation with a continued command line and the four pattern results.

LECTURE 4 | EXPERIMENT 35

Redirect input output and errors

Objective. Write and append output, read a file as input, create a here-document and handle standard error.

Commands used

```
printf 'Linux\nShell\n' > output.txt
printf 'Administration\n' >> output.txt
tr 'a-z' 'A-Z' < output.txt
cat missing_lab_file.txt 2> errors.txt
ls source/notes.txt missing_lab_file.txt > combined.txt 2>&1
printf 'Message sent to stderr\n' >&2
```

Result. The file first contained Linux and Shell, then Administration was appended. Input redirection fed those lines to tr. A here-document created a two-line file. The missing-file message was captured in errors.txt; combined.txt contained both the listing and the error. The final printf wrote directly to descriptor 2.

Standard descriptors are 0 for input, 1 for output and 2 for error. In > combined.txt 2>&1, the order directs both output channels to the same file. The missing-file errors were deliberately generated.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ printf 'Linux\nShell\n' > output.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat output.txt
Linux
Shell
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ printf 'Administration\n' >> output.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat output.txt
Linux
Shell
Administration
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ tr 'a-z' 'A-Z' < output.txt
LINUX
SHELL
ADMINISTRATION
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat << EOF > heredoc.txt
First line from a here-document
Second line from a here-document
EOF
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat heredoc.txt
First line from a here-document
Second line from a here-document
```

Figure 53. Output replacement, append, input redirection and here-document results.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat missing_lab_file.txt 2> errors.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat errors.txt
cat: missing_lab_file.txt: No such file or directory
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ls source/notes.txt missing_lab_file.txt > combined.txt 2
>&1
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ cat combined.txt
ls: cannot access 'missing_lab_file.txt': No such file or directory
source/notes.txt
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ printf 'Message sent to stderr\n' >&2
Message sent to stderr
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$
```

Figure 54. Separate error capture, merged output and an explicit write to stderr.

LECTURE 4 | EXPERIMENT 36

Use quoting and command substitution

Objective. Compare literal and expanded values, preserve spaces in an argument and insert command output into strings.

Commands used

```
lab_word="Linux"
echo '$lab_word'
echo "$lab_word"
echo \$lab_word
printf '<%s>\n' two words
printf '<%s>\n' "two words"
echo "Current location: $(pwd)"
lab_lines=$(wc -l < records.txt)
lab_today=`date +%F`
```

Result. Single quotes and the escaped dollar sign printed literal $lab_word, while double quotes allowed Linux to expand. Quoting two words kept them in one argument. A filename with spaces was created and listed. $(pwd), $(wc -l ...) and the backtick date substitution supplied the directory, record count 4 and date 2026-10-13.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ lab_word="Linux"
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo '$lab_word'
$lab_word
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "$lab_word"
Linux
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo \$lab_word
$lab_word
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ printf '<%s>\n' two words
<two>
<words>
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ printf '<%s>\n' "two words"
<two words>
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ touch "file with spaces.txt"
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ ls -l "file with spaces.txt"
-rw-rw-r-- 1 rgn rgn 0 Oct 13 19:35 'file with spaces.txt'
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ unset lab_word
```

Figure 55. Single and double quotes, escaping, argument grouping and a spaced filename.

```
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "Current location: $(pwd)"
Current location: /home/rgn/System_Admin_Lab
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ lab_lines=$(wc -l < records.txt)
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "Record lines: $lab_lines"
Record lines: 4
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ lab_today=`date +%F`
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ echo "Today: $lab_today"
Today: 2026-10-13
rgn@rgn-VMware-Virtual-Platform:~/System_Admin_Lab$ unset lab_lines lab_today
```

Figure 56. Dollar-parenthesis and backtick substitution with observed values.

ASSESSMENT OF RECORDED RESULTS

Results and conclusions

I completed a broad set of administration and shell exercises in my Ubuntu VM. The most useful checks were those that tested the outcome rather than only displaying a command: I restored an archive and compared checksums, observed cron-generated files, logged in as a new account, verified its removal, checked actual HTTP responses and tested denied access from another user.

Test	Observed outcome
Backup recovery	Restored notes.txt matched the original SHA-256 checksum.
Scheduled execution	Consecutive minute backups were recorded during the cron test.
Final backup configuration	Daily 18:00 entry installed; a daily run was not captured.
Account lifecycle	UID 1001 account created and tested; password changed; account and home removed.
Network tests	Gateway and external ping returned 0% loss; DNS resolved example.com.
Web service	HEAD and GET returned 200; both appeared in the log; process stopped.
Access policy	Owner read succeeded; nobody read returned Permission denied.
System health inspection	0 failed system units; journal warnings and user-session errors remained visible.
Post-install settings	Clock synchronised and NTP active; UFW inactive.

What I learned

I learned to pair an action with a verification command. File listings confirm structure, cat confirms readable contents, checksums compare recovered data, process and socket commands confirm a service is running, and logs show what it actually handled. Shell redirection, quoting and export also change how commands receive data and how child processes see variables.

Limits of this report

The evidence is from one existing Ubuntu VM. It does not establish complete software patching, a deployed firewall policy, a new OS installation, broad hardware compatibility testing or an automated multi-machine deployment. The remaining evidence checklist follows.

COVERAGE REVIEW

Activities not yet verified

These items were not verified in the supplied student captures. Part B now covers them through labelled educational simulations S01 to S26. The table describes the status of actual execution; it does not mean the topics are omitted from this supplemented report.

Lecture	Item	Evidence status
1	Automated user provisioning; adding or removing hardware; testing software across multiple OS and hardware platforms.	Not recorded.
1	Live wiki and request or ticket system deployment; recurring security audit beyond the permission test.	Local documentation only; no deployed platform or full security audit.
2	New VM creation and installer media selection; hardware compatibility or driver validation.	Existing guest inventory only.
2	Installer language, keyboard, partition creation, mount-point selection and boot-loader setup.	No installer screenshots.
2	Root-account policy during installation; initial user and installer network setup.	Account administration after installation is evidenced.
2	Firewall enabling and permitted-service rule tests; completed system patching; reboot and clean shutdown.	Inspected only or not recorded.
2	Dual boot; network or PXE installation; golden-image cloning; automated deployment.	No execution evidence.
3	man -K full-text manual search; native text-console login; X-specific remote applications.	Not recorded.
3	Interactive less output.	Launch-only capture excluded; no pager-output verification.
4	Arrow-key editing; Ctrl+A and Ctrl+E; Tab completion; use of other shell families.	Not separately captured.

The supplied captures are sufficient to document the experiments in this report. They are not a basis for labelling every practical topic in all four lectures as fully completed.

FURTHER EVIDENCE CHECKLIST

Documenting a new OS installation

The following sequence is a completion checklist for a separate installation exercise. It is not a record of work already executed. Use a new disposable VM so that the existing lab files remain available.

Stage	Evidence to capture
1 Plan the VM	VM name, guest OS choice, memory, CPU allocation, virtual disk capacity and selected ISO.
2 Boot the installer	Installer start screen; language and keyboard selections.
3 Configure storage	Selected virtual disk, partition layout, filesystem types and mount points; chosen swap arrangement.
4 Configure access	User and hostname choices; ordinary-user administration policy; network configuration. Do not include passwords.
5 Finish installation	Installation completion, first boot and successful login.
6 Verify the installed guest	OS and kernel output; disk and mount inventory; active network; boot and service logs.
7 Apply post-install settings	Time zone and NTP; tested firewall rules appropriate to required services; completed update results.
8 Finish and replicate	Clean reboot or shutdown evidence. If required, a clone or automation exercise with a clearly identified source and destination.

Useful verification commands for the new guest

```
cat /etc/os-release
uname -r
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
findmnt /
swapon --show
ip -br addr
ip route
systemctl --failed --no-pager
sudo journalctl -b -p warning -n 20 --no-pager
sudo ufw status verbose
timedatectl status
```

These commands support future collection of actual evidence. Part B supplies clearly labelled expected model results for installation topics. Ubuntu root-login and swap arrangements can differ from the older Fedora examples.

REFERENCE MATERIAL

Experiment index

No	Lecture	Page	Experiment
01	1	3	Hardware inventory
02	1	4	Create and test a user account
03	1	5	Change a password and remove the lab account
04	1	6	Create and restore a backup
05	1	7	Automate backups with a Bash script
06	1	8	Verify the cron service
07	1	9	Test scheduled backups with cron
08	1	10	Set the final daily backup schedule
09	1	11	Monitor resources and processes
10	1	12	Inspect the network configuration
11	1	13	Test connectivity and name resolution
12	1	14	Install curl and start a local web service
13	1	15	Verify HTTP responses and review web logs
14	1	16	Apply an owner only access policy
15	1	17	Document changes and recovery instructions
16	2	18	Identify the installed operating system
17	2	19	Inspect disks filesystems and free space
18	2	20	Check services logs firewall and time
19	3	21	Create and list files and directories
20	3	22	Copy move and rename files
21	3	23	Navigate paths and remove test files
22	3	24	Inspect and sort structured text
23	3	25	Filter merge and transform text
24	3	26	Read a command manual
25	3	27	Search manuals and inspect the hierarchy
26	3	28	View a long file with more
27	3	29	Use date echo and the calendar
28	3	30	Edit and save a file with vi
29	4	31	Identify Bash and use command history
30	4	32	Use aliases and case sensitive variables
31	4	33	Export variables and inspect the environment
32	4	34	Inspect startup files and source a configuration
33	4	35	Use logical operators background jobs and pipes
34	4	36	Expand filename patterns
35	4	37	Redirect input output and errors
36	4	38	Use quoting and command substitution

REFERENCE MATERIAL

Sources and evidence notes

Course sources

Lecture1-Intro(2).ppt - Introduction to System and Network Administration.
Lecture2-OS-Install(2).ppt - OS Installing.
Lecture3-Intro-To-Linux(1).ppt - Command Line Interface.
Lecture4-Shells.ppt - Shell Basics.

The lecture files identify course 10686341 Network Administration and credit Scott Orr and the Trustees of Indiana University. Their topic lists were used to organise the practical evidence and identify the coverage gaps. This report uses current observed Ubuntu outputs rather than copying the old Fedora installation requirements as present-day recommendations.

Practical evidence

Source: the student-supplied Word document of 59 terminal and file-manager screenshots from the practical session on 13 October 2026. Selected images were extracted and inserted without replacing their command output. Earlier uploads from the same session were used to clarify the sequence and results.

Capture selection

Omitted captures were the repeated creation display for the documentation files, the vi launch command by itself, the man launch command by itself and the less launch command by itself. The actual vi text and saved output, the displayed manual, and the saved documentation contents were retained. Some figures are intentionally reused where one capture verifies both an account task and scheduled backups.

Interpretation rules

Installed package status is different from complete system patching. A configured daily cron entry is different from a captured daily run. An existing installed system is different from a documented fresh installation. A zero failed-unit system query is different from an error-free journal. These distinctions keep the report aligned with the supplied evidence.

Final recorded state

The temporary user account had been removed. The local HTTP test server had been stopped. The backup script remained in the lab directory and the user crontab scheduled it daily at 18:00. Local documentation and permission-test files remained in System_Admin_Lab. UFW was inactive when inspected; the system clock was synchronised.

PART B | EDUCATIONAL SIMULATIONS

Completion through illustrated simulations

This supplement provides model applications for topics not documented in the student screenshots. Every new picture is labelled SIMULATION. Outcomes are expected or illustrative results, not additional student execution. Part A retains the actual practical captures.

Model environment

The separate hypothetical Ubuntu guest is sim-vm, its administration user is labadmin and its blank main disk is 40 GiB. Spare disks and image files belong only to the model. Private addresses identify an isolated installation network. These names and values do not describe modifications to the student VM.

Coverage added

The models cover automated accounts, hardware recognition, compatibility tests, documentation and requests, security review, VM creation, media, installer choices, partitioning and resizing, GRUB and authentication, user policy, network and time, firewall and patches, boot testing, network sources, PXE, automated installation, cloning, manual searches, paging, console login, line editing, X applications and other shells.

How to read a model

Each model includes an objective, procedure, command or configuration example and expected result. Square-bracketed text describes an interaction or shortened output, not a literal program message. Placeholders must be replaced when building a real environment. Multi-machine tasks specify their prerequisite server, client or second operating system.

PART B | SIMULATION INDEX

Simulation activity index

Model	Lecture	Page	Topic
S01	1	46	Automate ordinary user provisioning
S02	1	47	Recognise added hardware and storage
S03	1	48	Test software across operating systems
S04	1	49	Maintain a wiki and help desk record
S05	1	50	Perform a periodic security review
S06	2	51	Create the installation virtual machine
S07	2	52	Verify media and choose an installation method
S08	2	53	Select language keyboard and install options
S09	2	54	Partition the disk and assign mount points
S10	2	55	Shrink a filesystem and its partition
S11	2	56	Configure GRUB and a dual boot menu
S12	2	57	Protect GRUB menu editing
S13	2	58	Set identity and least privilege
S14	2	59	Configure network and synchronised time
S15	2	60	Apply and test a firewall policy
S16	2	61	Install patches and verify update status
S17	2	62	Test first boot reboot and clean shutdown
S18	2	63	Compare HTTP NFS and FTP sources
S19	2	64	Boot an installer through UEFI PXE
S20	2	65	Automate an Ubuntu installation
S21	2	66	Create and validate a golden image clone
S22	3	67	Search manual contents and use less
S23	3	68	Use a native text console login
S24	4	69	Edit command lines and complete pathnames
S25	3	70	Run a remote X application through SSH
S26	4	71	Compare shell families and portability

PART B | LECTURE 1 | S01 SIMULATION

Automate ordinary user provisioning

Objective. Model repeatable user creation.

Procedure. Save the script below, run it as root in the model guest and inspect its accounts. After verification, remove only sim_user01 and sim_user02.

Command or configuration example

```
#!/bin/bash
set -e
for lab_user in sim_user01 sim_user02; do
    if ! id "$lab_user" >/dev/null 2>&1; then
        useradd -m -s /bin/bash "$lab_user"
    fi
    id "$lab_user"
done
# Run: sudo bash provision_lab.sh
# Cleanup: sudo userdel -r sim_user01; sudo userdel -r sim_user02
```

Expected model result. The model creates two ordinary users and home directories without adding sudo membership. Re-running the script detects existing accounts. useradd accounts need an authentication method before interactive login.

S01 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
$ sudo bash provision_lab.sh
uid=1201(sim_user01) gid=1201(sim_user01) groups=1201(sim_user01)
uid=1202(sim_user02) gid=1202(sim_user02) groups=1202(sim_user02)
$ getent passwd sim_user01
sim_user01:x:1201:1201::/home/sim_user01:/bin/bash
$ sudo userdel -r sim_user01
$ sudo userdel -r sim_user02
$ getent passwd sim_user01 || echo "Lab account removed"
Lab account removed
```

Simulation figure 01. Automate ordinary user provisioning. Educational model.

PART B | LECTURE 1 | S02 SIMULATION

Recognise added hardware and storage

Objective. Model a spare virtual disk and driver recognition.

Procedure. Power off the model VM, attach a new blank 10 GiB disk and boot. Identify it as /dev/vdb, create its filesystem and mount it. Check the driver reported for the network controller.

Command or configuration example

```
lspci -nnk | grep -A 3 -i ethernet
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
sudo parted -s /dev/vdb mklabel gpt
sudo parted -s /dev/vdb mkpart primary ext4 1MiB 100%
sudo mkfs.ext4 /dev/vdb1
sudo mkdir -p /mnt/sim_data
sudo mount /dev/vdb1 /mnt/sim_data
findmnt /mnt/sim_data
sudo umount /mnt/sim_data
```

Expected model result. The guest recognises the model disk and mounts ext4. To remove it, unmount, power off and detach it. This example never formats the student VM disk.

S02 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
$ lspci -nnk | grep -A 3 -i ethernet
03:00.0 Ethernet controller: Red Hat Virtio network device
        Kernel driver in use: virtio-pci
$ lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
NAME   SIZE TYPE FSTYPE MOUNTPOINTS
vda     40G disk
vdb     10G disk
$ findmnt /mnt/sim_data
TARGET        SOURCE    FSTYPE OPTIONS
/mnt/sim_data /dev/vdb1 ext4   rw,relatime
[Model VM later powered off; spare disk detached]
```

Simulation figure 02. Recognise added hardware and storage. Educational model.

PART B | LECTURE 1 | S03 SIMULATION

Test software across operating systems

Objective. Model a small compatibility comparison.

Procedure. Use Ubuntu and Fedora guests with different CPU and memory allocations. Install curl with the corresponding package manager. Run a Python loopback web server in each guest and check its response in another terminal.

Command or configuration example

```
# Ubuntu: sudo apt install -y curl
# Fedora: sudo dnf install -y curl
mkdir -p sim_web
echo "Compatibility test" > sim_web/index.html
python3 -m http.server 8080 --bind 127.0.0.1 --directory sim_web
# In another terminal in each guest:
curl -I http://127.0.0.1:8080/
```

Expected model result. Both model guests return HTTP 200 in the sample matrix. This illustrates a basic software test, not comprehensive hardware certification. Record actual versions and resources in a real comparison.

S03 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
SIMULATED COMPATIBILITY REGISTER
Guest     CPU profile  RAM     Package tool   HTTP result
Ubuntu    2 vCPU       4 GiB   APT            200 OK
Fedora    4 vCPU       6 GiB   DNF            200 OK
$ curl -I http://127.0.0.1:8080/
HTTP/1.0 200 OK
Content-type: text/html
Stop each test server after checking the response.
```

Simulation figure 03. Test software across operating systems. Educational model.

PART B | LECTURE 1 | S04 SIMULATION

Maintain a wiki and help desk record

Objective. Model a documentation and request workflow.

Procedure. Create a small operations page and CSV request register, serve the directory locally and retrieve the page. Close the model backup request after documenting its checksum verification.

Command or configuration example

```
mkdir -p sim_docs
echo '<h1>Recovery</h1><p>Extract and compare checksums.</p>' > sim_docs/index.html
printf 'id,request,status,evidence\nSIM-001,Verify restore,resolved,matching  \
checksums\n' > sim_docs/tickets.csv
python3 -m http.server 8000 --bind 127.0.0.1 --directory sim_docs
# In a second terminal:
curl http://127.0.0.1:8000/
cat sim_docs/tickets.csv
```

Expected model result. The model demonstrates a browsable documentation page and a traceable request entry using static HTML and CSV. It does not claim installation of a production wiki or help-desk product.

S04 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
$ curl http://127.0.0.1:8000/
<h1>Recovery</h1><p>Extract and compare checksums.</p>
$ cat sim_docs/tickets.csv
id,request,status,evidence
SIM-001,Verify restore,resolved,matching checksums
MODEL WORKFLOW
Request opened -> recovery checked -> evidence recorded -> resolved
Stop the documentation server after the model test.
```

Simulation figure 04. Maintain a wiki and help desk record. Educational model.

PART B | LECTURE 1 | S05 SIMULATION

Perform a periodic security review

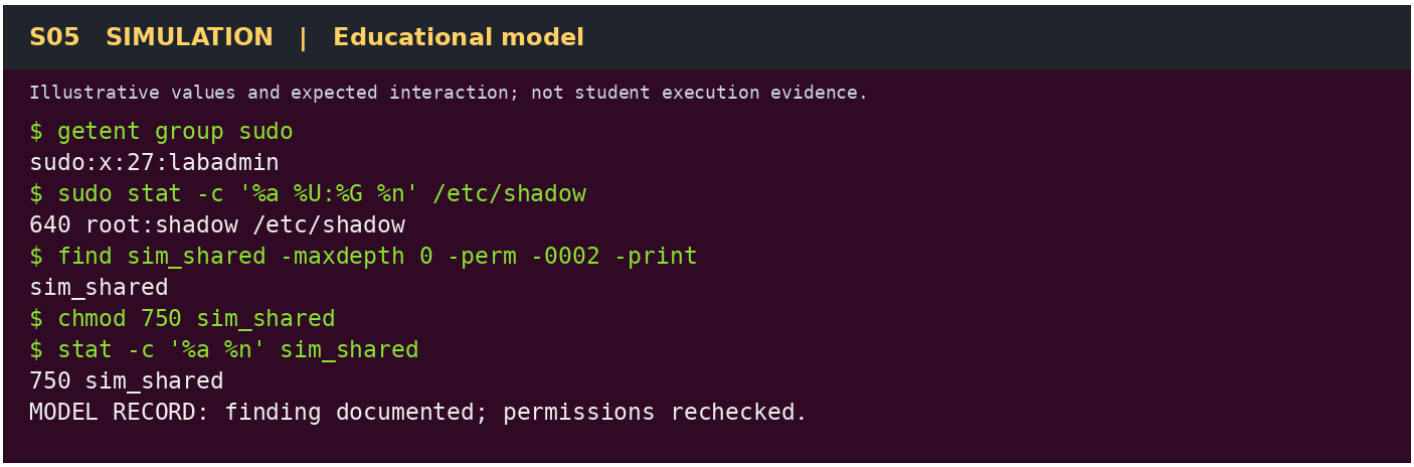
Objective. Model reviewing privileges permissions and logs.

Procedure. Inspect sudo membership, shadow-file permissions and authentication messages. Find the intentionally world-writable model directory, apply the local policy and record the finding.

Command or configuration example

```
getent group sudo
sudo stat -c '%a %U:%G %n' /etc/shadow
sudo journalctl -b -t sshd -n 10 --no-pager
mkdir -p sim_shared
chmod 777 sim_shared
find sim_shared -maxdepth 0 -perm -0002 -print
chmod 750 sim_shared
stat -c '%a %n' sim_shared
```

Expected model result. The model detects and corrects a writable-by-everyone lab directory. Short log excerpts and permission checks support a review but cannot prove the absence of compromise.



Simulation figure 05. Perform a periodic security review. Educational model.

PART B | LECTURE 2 | S06 SIMULATION

Create the installation virtual machine

Objective. Model the New and Build lifecycle stages.

Procedure. Create a separate guest in the hypervisor wizard, allocate resources, attach the official amd64 ISO, choose UEFI and boot. The view below is schematic, not a literal VMware screen.

Command or configuration example

```
# Name: System_Admin_Install_Sim
# Guest: Ubuntu Server LTS, amd64
# CPU: 2 vCPU; RAM: 4 GiB; virtual disk: 40 GiB
# UEFI; isolated installation network; ISO connected
```

Expected model result. The model has an independent blank virtual disk. Compare CPU, memory, disk, graphics and networking requirements with the chosen vendor release before an actual installation.

Reference: Vendor documentation.

S06 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
MODEL VIRTUAL MACHINE CONFIGURATION
Name                System_Admin_Install_Sim
Operating system    Ubuntu Server LTS, amd64
Processors          2 vCPU
Memory              4096 MiB
Disk                40 GiB, blank model disk
Firmware            UEFI
Optical drive       Ubuntu Server ISO connected
Network             Isolated lab / optional NAT
Lifecycle           New -> Build -> Clean
Next                Boot installation media
```

Simulation figure 06. Create the installation virtual machine. Educational model.

PART B | LECTURE 2 | S07 SIMULATION

Verify media and choose an installation method

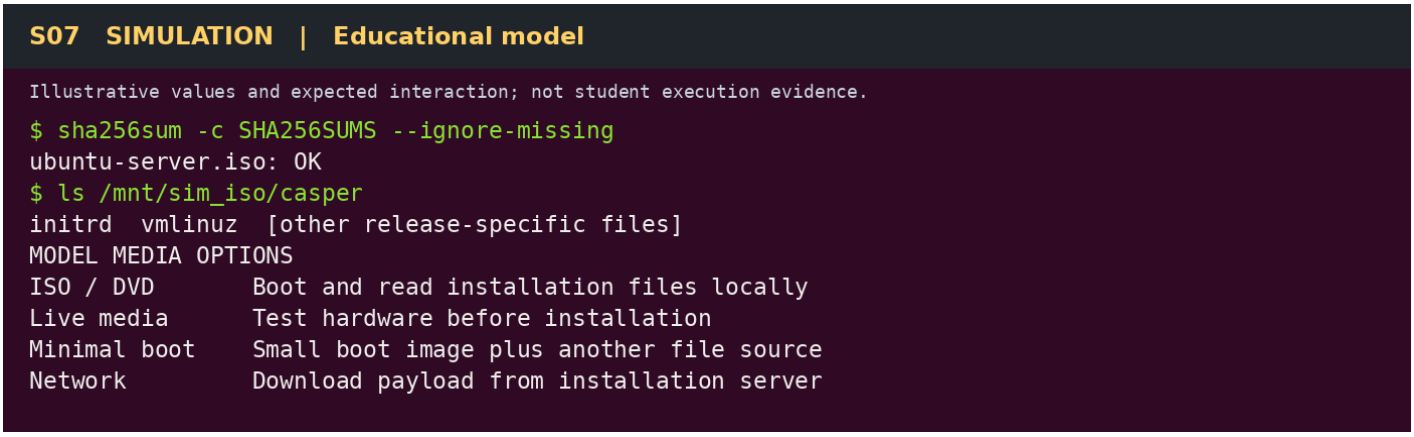
Objective. Model media validation and source selection.

Procedure. Obtain vendor media and authenticated checksum information. Compare the ISO checksum, inspect the read-only media and choose local or network installation.

Command or configuration example

```
cd ~/Downloads
sha256sum -c SHA256SUMS --ignore-missing
sudo mkdir -p /mnt/sim_iso
sudo mount -o loop,ro ubuntu-server.iso /mnt/sim_iso
ls /mnt/sim_iso/casper
sudo umount /mnt/sim_iso
```

Expected model result. The illustrated checksum returns OK. The sample filename is a local alias. Live media can test a machine, full ISO or DVD media provide files locally, and minimal boot media require an additional payload source.



Simulation figure 07. Verify media and choose an installation method. Educational model.

PART B | LECTURE 2 | S08 SIMULATION

Select language keyboard and install options

Objective. Model early interactive installer choices.

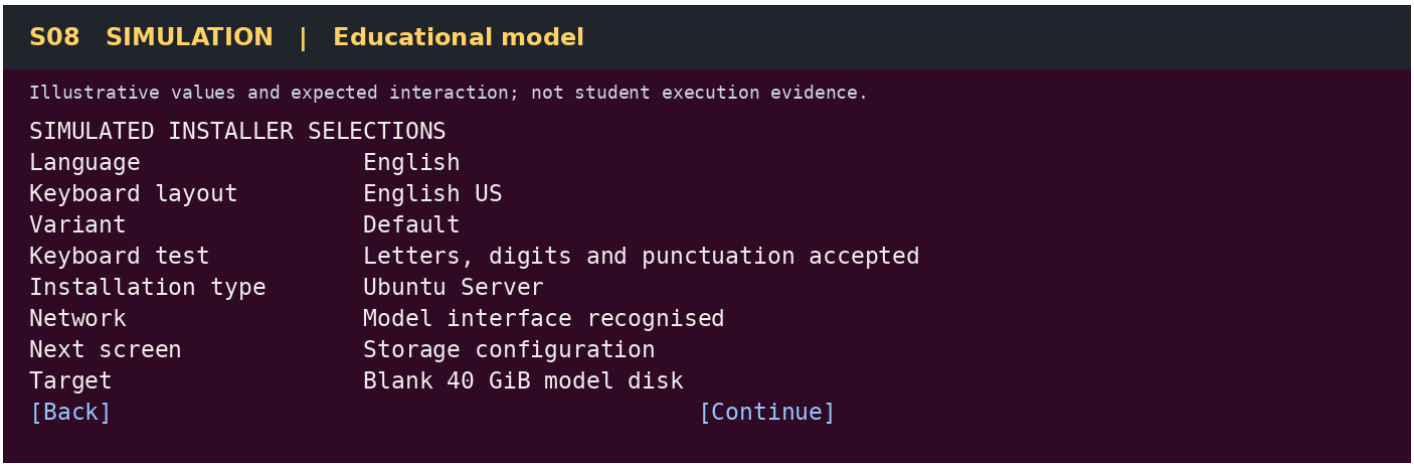
Procedure. Choose English and the US keyboard layout, test punctuation and select the server install. Confirm the recognised interface before moving to storage.

Command or configuration example

```
# Installer sequence:
# Language -> Keyboard -> Install type -> Network -> Storage
```

Expected model result. The schematic records language and input choices. Actual screen layouts and available options depend on the installer release.

Reference: Vendor documentation.



Simulation figure 08. Select language keyboard and install options. Educational model.

PART B | LECTURE 2 | S09 SIMULATION

Partition the disk and assign mount points

Objective. Model a UEFI storage layout.

Procedure. Select the blank model disk and GPT. Create an EFI partition, root, swap and home; assign mount points and confirm the installer summary. Inspect the resulting hypothetical first boot.

Command or configuration example

```
lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINTS
findmnt /
findmnt /home
findmnt /boot/efi
swapon --show
```

Expected model result. The study layout uses 512 MiB EFI, 30 GiB root, 4 GiB swap and 5 GiB home, leaving roughly 0.5 GiB for alignment and unused space. These allocations are examples, not vendor minimums or changes to the original VM.

S09 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
$ lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINTS
NAME     SIZE FSTYPE MOUNTPOINTS
vda       40G
vda1     512M vfat   /boot/efi
vda2      30G ext4   /
vda3       4G swap   [SWAP]
vda4       5G ext4   /home
$ findmnt /
TARGET SOURCE    FSTYPE OPTIONS
/      /dev/vda2 ext4   rw,relatime
$ swapon --show
NAME      TYPE      SIZE USED PRIO
/dev/vda3 partition   4G   0B   -2
```

Simulation figure 09. Partition the disk and assign mount points. Educational model.

PART B | LECTURE 2 | S10 SIMULATION

Shrink a filesystem and its partition

Objective. Model retaining data during ext4 shrinking.

Procedure. Create a disposable sparse image and loop device. Format its first partition and write keep.txt. Unmount it, check and shrink the filesystem before reducing the partition. Add a second partition, remount and verify the file.

Command or configuration example

```
truncate -s 10G sim_disk.img
lab_loop=$(sudo losetup --find --show --partscan sim_disk.img)
lab_partition="$lab_loop"p1
sudo parted -s "$lab_loop" mklabel gpt mkpart primary ext4 1MiB 9GiB
sudo partprobe "$lab_loop"
sudo mkfs.ext4 "$lab_partition"
# Mount, write keep.txt, then unmount before resizing.
sudo e2fsck -f "$lab_partition"
sudo resize2fs "$lab_partition" 4G
sudo parted -s "$lab_loop" resizepart 1 5GiB
sudo parted -s "$lab_loop" mkpart primary ext4 5GiB 9GiB
sudo partprobe "$lab_loop"
# Remount and verify; unmount and detach loop device afterwards.
```

Expected model result. The model retains a 4 GiB ext4 filesystem inside the reduced first partition. The filesystem must be unmounted for this shrink, and its new size must fit the new partition. This is not a procedure for every filesystem.

S10 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
SIMULATED RESIZE CHECKPOINTS
1 Disposable 10 GiB image created
2 First filesystem formatted; keep.txt written
3 Filesystem unmounted and checked
4 ext4 filesystem reduced to 4 GiB
5 First partition end reduced to 5 GiB
6 Second partition created in the freed space
$ cat /mnt/sim_resize/keep.txt
Keep this lab data
MODEL RESULT: original test contents retained.
```

Simulation figure 10. Shrink a filesystem and its partition. Educational model.

PART B | LECTURE 2 | S11 SIMULATION

Configure GRUB and a dual boot menu

Objective. Model boot-loader settings and OS discovery.

Procedure. Assume a Windows installation exists on a second model disk. Install os-prober, edit the listed variables and regenerate GRUB. Inspect the menu entries.

Command or configuration example

```
sudo apt install -y os-prober
sudoedit /etc/default/grub
# GRUB_TIMEOUT_STYLE=menu
# GRUB_TIMEOUT=5
# GRUB_DISABLE_OS_PROBER=false
sudo update-grub
grep -E "^menuentry " /boot/grub/grub.cfg
```

Expected model result. The model menu offers Ubuntu and Windows Boot Manager. Generating a menu does not install a second OS or prove that both systems actually boot. OS discovery depends on firmware and disk contents.

Reference: Vendor documentation.

S11 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
$ sudo update-grub
[Illustrative excerpt]
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-[installed kernel]
Found Windows Boot Manager on /dev/vdb1
done
SIMULATED BOOT MENU
Ubuntu
Advanced options for Ubuntu
Windows Boot Manager
UEFI Firmware Settings
```

Simulation figure 11. Configure GRUB and a dual boot menu. Educational model.

PART B | LECTURE 2 | S12 SIMULATION

Protect GRUB menu editing

Objective. Model boot-menu authentication.

Procedure. Generate a PBKDF2 hash interactively. Add the superuser and complete generated hash to the custom GRUB script, regenerate the configuration and test an attempted edit. Choose whether normal boot entries should be unrestricted.

Command or configuration example

```
grub-mkpasswd-pbkdf2
sudoedit /etc/grub.d/40_custom
# Below the existing script header add:
# set superusers="instructor"
# password_pbkdf2 instructor COMPLETE_GENERATED_HASH
sudo update-grub
# At the model menu: press e and test authentication.
```

Expected model result. The model requests authentication before a protected edit. The placeholder is not a valid hash: an implementation must insert the generated value. No password or student boot configuration was changed.

Reference: Vendor documentation.

S12 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
$ grub-mkpasswd-pbkdf2
Enter password: [hidden]
Reenter password: [hidden]
PBKDF2 hash of your password is grub.pbkdf2.sha512.[full hash]
SIMULATED GRUB TEST
Request: edit protected entry
Enter username: instructor
Enter password: [hidden]
Authentication accepted
Boot-entry editor opened
```

Simulation figure 12. Protect GRUB menu editing. Educational model.

PART B | LECTURE 2 | S13 SIMULATION

Set identity and least privilege

Objective. Model first-user and root policy.

Procedure. Choose labadmin and hostname sim-vm in the installer. After first boot, inspect sudo membership and root password status, create an ordinary user and compare its privileges.

Command or configuration example

```
hostnamectl
id labadmin
sudo passwd -S root
sudo adduser sim_guest
id sim_guest
sudo -l -U sim_guest
```

Expected model result. The model administrator belongs to sudo, while the ordinary account does not. A locked root password prevents root password login but does not prevent authorised sudo operations.

S13 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
$ id labadmin
uid=1000(labadmin) gid=1000(labadmin) groups=1000(labadmin),27(sudo)
$ sudo passwd -S root
root L [date] 0 99999 7 -1
$ id sim_guest
uid=1001(sim_guest) gid=1001(sim_guest) groups=1001(sim_guest),100(users)
$ sudo -l -U sim_guest
User sim_guest is not allowed to run sudo on sim-vm.
Installer password: entered privately, never included.
```

Simulation figure 13. Set identity and least privilege. Educational model.

PART B | LECTURE 2 | S14 SIMULATION

Configure network and synchronised time

Objective. Model initial networking and time settings.

Procedure. Save the model YAML in /etc/netplan/99-sim-lab.yaml with mode 600. Confirm the interface name, apply it with netplan try, then set the time zone and enable NTP. The lab must provide gateway, DNS and time access.

Command or configuration example

```
network:
  version: 2
  ethernets:
    ens3:
      dhcp4: false
      addresses: [192.168.56.20/24]
      routes:
        - to: default
          via: 192.168.56.1
      nameservers:
        addresses: [192.168.56.1]
# sudo netplan try
# sudo timedatectl set-timezone Asia/Hebron
# sudo timedatectl set-ntp true
```

Expected model result. The model has address .20 and a default route through .1. These hypothetical values do not replace the student network. Clock synchronisation requires a reachable configured time source.

S14 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
$ ip -br addr
lo     UNKNOWN 127.0.0.1/8 ::1/128
ens3   UP      192.168.56.20/24
$ ip route
default via 192.168.56.1 dev ens3
192.168.56.0/24 dev ens3 scope link
$ timedatectl status
Time zone: Asia/Hebron
System clock synchronized: yes
NTP service: active
```

Simulation figure 14. Configure network and synchronised time. Educational model.

PART B | LECTURE 2 | S15 SIMULATION

Apply and test a firewall policy

Objective. Model enabling only intended incoming services.

Procedure. Allow management SSH before enabling the model firewall. Permit HTTP, deny Telnet and inspect the policy. Test from a second model guest with services known to be listening.

Command or configuration example

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 192.168.56.0/24 to any port 22 proto tcp
sudo ufw allow 80/tcp
sudo ufw deny 23/tcp
sudo ufw logging on
sudo ufw enable
sudo ufw status verbose
# Peer: curl -I http://192.168.56.20/
# Peer: ssh labadmin@192.168.56.20
```

Expected model result. The model firewall is active. HTTP success assumes a listening web service. A failed connection alone cannot distinguish a firewall rule from a stopped service; compare sockets and logs. The actual student UFW observation remains inactive.

Reference: Vendor documentation.

S15 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
To         Action    From
22/tcp     ALLOW IN  192.168.56.0/24
80/tcp     ALLOW IN  Anywhere
23/tcp     DENY IN   Anywhere
[Model peer; HTTP server is listening]
$ curl -I http://192.168.56.20/
HTTP/1.1 200 OK
This clean model guest is separate from the IPSec lab VM.
```

Simulation figure 15. Apply and test a firewall policy. Educational model.

PART B | LECTURE 2 | S16 SIMULATION

Install patches and verify update status

Objective. Model patch installation and reboot checks.

Procedure. Refresh metadata, review upgradeable packages and install available upgrades. List them again, inspect the reboot indicator and check the package history.

Command or configuration example

```
sudo apt update
apt list --upgradable
sudo apt upgrade
apt list --upgradable
if test -f /var/run/reboot-required; then
    cat /var/run/reboot-required
fi
sudo tail -n 12 /var/log/apt/history.log
```

Expected model result. The sample sequence leaves no upgradeable entries and shows a reboot request after a model kernel update. Real counts depend on repository state. The original student captures still show packages not upgraded.

S16 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
$ apt list --upgradable
Listing... Done
[Model packages available before updating]
$ sudo apt upgrade
[Model update transaction completes]
$ apt list --upgradable
Listing... Done
$ cat /var/run/reboot-required
*** System restart required ***
$ sudo tail -n 12 /var/log/apt/history.log
[Completed model upgrade transaction recorded]
```

Simulation figure 16. Install patches and verify update status. Educational model.

PART B | LECTURE 2 | S17 SIMULATION

Test first boot reboot and clean shutdown

Objective. Model boot validation and lifecycle transitions.

Procedure. Check login, services and warning messages. Record the boot ID, request reboot and compare it after login. Request clean shutdown and confirm the model hypervisor shows the guest powered off.

Command or configuration example

```
whoami
systemctl --failed --no-pager
sudo journalctl -b -p warning --no-pager
cat /proc/sys/kernel/random/boot_id
sudo reboot
# After a new login:
cat /proc/sys/kernel/random/boot_id
sudo shutdown -h now
```

Expected model result. The model accepts a login and reports a different boot ID after reboot. Shutdown terminates services and unmounts filesystems before the guest powers off. The displayed IDs are deliberately illustrative.

S17 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
sim-vm login: labadmin
Password: [hidden]
$ whoami
labadmin
$ systemctl --failed --no-pager
0 loaded units listed.
$ cat /proc/sys/kernel/random/boot_id
11111111-1111-4111-8111-111111111111
$ sudo reboot
[Guest restarts; new login]
$ cat /proc/sys/kernel/random/boot_id
22222222-2222-4222-8222-222222222222
$ sudo shutdown -h now
[Model hypervisor: Powered off]
```

Simulation figure 17. Test first boot reboot and clean shutdown. Educational model.

PART B | LECTURE 2 | S18 SIMULATION

Compare HTTP NFS and FTP sources

Objective. Model installation payload availability.

Procedure. Prepare a read-only installation tree on a model server. Probe HTTP, list an NFS export, mount it read-only and list an FTP directory. Choose a source supported by the selected installer.

Command or configuration example

```
curl -I http://192.168.56.10/images/ubuntu-server.iso
showmount -e 192.168.56.10
sudo mkdir -p /mnt/sim_repo
sudo mount -t nfs -o ro 192.168.56.10:/srv/install /mnt/sim_repo
ls /mnt/sim_repo
sudo umount /mnt/sim_repo
curl ftp://192.168.56.10/install/
```

Expected model result. The model confirms file availability. NFS and FTP preserve the historical lecture examples; these probes do not imply that modern Ubuntu accepts each as a native installer repository. Server services and client tools are assumed configured.

S18 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
$ curl -I http://192.168.56.10/images/ubuntu-server.iso
HTTP/1.1 200 OK
Content-Type: application/octet-stream
$ showmount -e 192.168.56.10
Export list for 192.168.56.10:
/srv/install 192.168.56.0/24
$ ls /mnt/sim_repo
images  Packages  repodata
$ curl ftp://192.168.56.10/install/
[Read-only model directory listing]
images/
Packages/
repodata/
```

Simulation figure 18. Compare HTTP NFS and FTP sources. Educational model.

PART B | LECTURE 2 | S19 SIMULATION

Boot an installer through UEFI PXE

Objective. Model DHCP TFTP and HTTP roles.

Procedure. On an isolated network configure the DHCP UEFI boot-file response. Serve a signed GRUB EFI file, matching kernel and initrd over TFTP, and the ISO over HTTP. Select the model GRUB entry.

Command or configuration example

```
# TFTP: grubnetx64.efi.signed, grub/grub.cfg,
# casper/vmlinuz and casper/initrd from the target ISO.
# DHCP supplies the TFTP server and UEFI boot-file name.
menuentry "Install Ubuntu from model server" {
    set gfxpayload=keep
    linux /casper/vmlinuz ip=dhcp url=http://192.168.56.10/images/ubuntu-server.iso  \
---
    initrd /casper/initrd
}
# Check: systemctl is-active tftpd-hpa
# Check the HTTP ISO URL with curl -I.
```

Expected model result. The model client obtains an address and loads the installer. All boot files must match the target release. DHCP and TFTP setup, firmware and Secure Boot support are prerequisites; this is not a student-network packet capture.

Reference: Vendor documentation.

S19 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
SIMULATED UEFI PXE TRACE
1 Client sends DHCP discovery
2 DHCP supplies address 192.168.56.30 and UEFI boot file
3 TFTP serves grubnetx64.efi.signed
4 GRUB loads its network boot menu
5 TFTP serves casper/vmlinuz and casper/initrd
6 HTTP serves images/ubuntu-server.iso
7 Installer displays its language screen
Server: 192.168.56.10
DHCP: UDP 67/68   TFTP: UDP 69   HTTP: TCP 80
```

Simulation figure 19. Boot an installer through UEFI PXE. Educational model.

PART B | LECTURE 2 | S20 SIMULATION

Automate an Ubuntu installation

Objective. Model autoinstall and a NoCloud seed.

Procedure. Generate a password hash with openssl passwd -6 and put it in the YAML below, saved as user-data. Create empty meta-data and seed.iso with cloud-localds. Attach the seed, blank disk and matching ISO to the model VM, and supply autoinstall on the installer kernel command line.

Command or configuration example

```
#cloud-config
autoinstall:
  version: 1
  identity:
    hostname: sim-auto01
    username: labadmin
    password: "REPLACE_WITH_GENERATED_PASSWORD_HASH"
  locale: en_US.UTF-8
  keyboard:
    layout: us
  storage:
    layout:
      name: direct
  ssh:
    install-server: true
# touch meta-data
# cloud-localds seed.iso user-data meta-data
```

Expected model result. The model applies identity and storage automatically. The password is a template placeholder requiring a valid generated hash before implementation. Validate YAML and target only the intended blank disk. Kickstart and Windows deployment tools are alternative platform examples from the lecture.

Reference: Vendor documentation.

S20 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
SIMULATED AUTOINSTALL PROGRESS
NoCloud seed detected
user-data loaded and configuration accepted
Single blank model disk selected
Storage layout applied
System packages installed
labadmin and sim-auto01 configured
OpenSSH server installed
Installation completed; reboot requested
The placeholder must be replaced before real validation.
```

Simulation figure 20. Automate an Ubuntu installation. Educational model.

PART B | LECTURE 2 | S21 SIMULATION

Create and validate a golden image clone

Objective. Model replication of a configured guest.

Procedure. Test and power off the source, make a full clone in the hypervisor and regenerate virtual hardware identity. Prepare unique guest machine ID and SSH host keys, set the clone hostname and inspect its address and services.

Command or configuration example

```
# Hypervisor: powered-off source -> full clone -> new VM identity
sudo hostnamectl set-hostname sim-clone01
hostnamectl --static
ip -br addr
cat /etc/machine-id
systemctl --failed --no-pager
# Compare machine IDs and SSH host-key fingerprints with source.
```

Expected model result. The model preserves the software baseline while using distinct host and network identities. Cloning alone does not guarantee unique machine ID or SSH host keys; image preparation must handle them.

S21 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
SIMULATED CLONE REGISTER
Source              sim-golden, powered off
Clone               sim-clone01, full clone
Virtual MAC         Regenerated
Hostname            sim-clone01
IPv4 address        Distinct DHCP lease
Machine ID          Distinct after image preparation
SSH host keys       Regenerated during clone preparation
Services            Required baseline passes checks
Lifecycle           Configured image -> replicated guest
```

Simulation figure 21. Create and validate a golden image clone. Educational model.

PART B | LECTURE 3 | S22 SIMULATION

Search manual contents and use less

Objective. Model full-text search and interactive paging.

Procedure. Run the section-limited man -K search and select a matching manual. Exit with q. Open a numbered file in less, search for 50, use n for another match, g and G for the first and last lines, then quit.

Command or configuration example

```
man -K -s 1 "current directory"
seq 1 100 > sim_long.txt
less sim_long.txt
# In less: /50, Enter; n; g; G; q
```

Expected model result. man -K searches manual contents, whereas man -k searches short descriptions. The model covers pager navigation without claiming that the earlier student less launch recorded these actions.

Reference: Vendor documentation.

S22 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
$ man -K -s 1 "current directory"
[Matching section 1 manual selected and displayed]
[User presses q]
$ less sim_long.txt
[User types /50 and presses Enter]
50
51
52
[User presses G]
98
99
100
(END)
[User presses q; shell resumes]
```

Simulation figure 22. Search manual contents and use less. Educational model.

PART B | LECTURE 3 | S23 SIMULATION

Use a native text console login

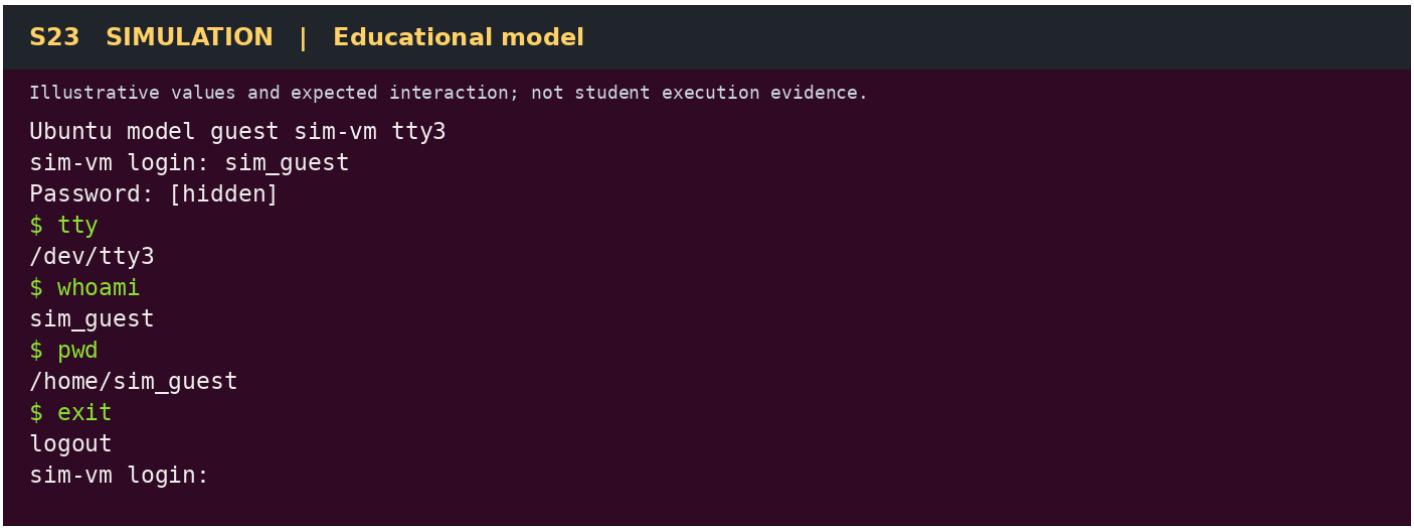
Objective. Model virtual-console login and logout.

Procedure. Switch to an available console such as Ctrl+Alt+F3. Log in with the model ordinary account, inspect the terminal and identity, then exit. Return to the GUI using its assigned virtual-terminal key.

Command or configuration example

```
# Enter username and password at the console login prompt.
tty
whoami
pwd
exit
```

Expected model result. The model /dev/tty3 is a native virtual console, unlike the actual /dev/pts/0 terminal emulator. The graphical session virtual-terminal number depends on the system.



Simulation figure 23. Use a native text console login. Educational model.

PART B | LECTURE 4 | S24 SIMULATION

Edit command lines and complete pathnames

Objective. Model keyboard editing and completion.

Procedure. Run echo, recall it with Up and move through history with Down. Edit using Left and Right, jump with Ctrl+A and Ctrl+E, then execute the changed line. Complete an unambiguous pathname with Tab.

Command or configuration example

```
echo history-demo-one
# Up/Down: command history; Left/Right: cursor movement
# Ctrl+A: beginning; Ctrl+E: end
# Edit one to two, then press Enter.
mkdir -p Simulation_Lab
# Type cd Simu, press Tab, then Enter.
pwd
```

Expected model result. The model repeats an edited command and completes Simulation_Lab. An ambiguous prefix requires more typing or another Tab to display alternatives.

S24 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
$ echo history-demo-one
history-demo-one
[Up] recalls echo history-demo-one
[Ctrl+A] cursor at beginning
[Ctrl+E] cursor at end
[Left/Right] move cursor to edit the final word
[Edit and Enter] echo history-demo-two
history-demo-two
$ cd Simu[Tab]
[Expanded: cd Simulation_Lab/]
$ pwd
/home/labadmin/Simulation_Lab
```

Simulation figure 24. Edit command lines and complete pathnames. Educational model.

PART B | LECTURE 3 | S25 SIMULATION

Run a remote X application through SSH

Objective. Model remote graphical application forwarding.

Procedure. Use two trusted model guests and an X-capable client. Install the server tools, ensure effective X11Forwarding yes, validate and reload SSH. Connect with -X, inspect DISPLAY and launch xclock.

Command or configuration example

```
# Model server:
sudo apt install -y openssh-server xauth x11-apps
sudoedit /etc/ssh/sshd_config
# Effective setting: X11Forwarding yes
sudo sshd -t
sudo systemctl reload ssh
# X-capable model client:
ssh -X labadmin@192.168.56.20
echo "$DISPLAY"
xclock
```

Expected model result. The model displays the remote clock on the client. It requires X or compatible Xwayland, remote xauth and SSH permissions; ordinary SSH without forwarding does not establish that display.

Reference: Vendor documentation.

S25 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
[Model graphical client]
$ ssh -X labadmin@192.168.56.20
[Authenticated remote session]
$ echo "$DISPLAY"
localhost:10.0
$ xclock
[Clock window appears on the model client]
Execution: remote guest
Display: local X-capable client
Transport: SSH X11 forwarding
Close the window, then exit the remote shell.
```

Simulation figure 25. Run a remote X application through SSH. Educational model.

PART B | LECTURE 4 | S26 SIMULATION

Compare shell families and portability

Objective. Model Bourne and C-shell syntax.

Procedure. Install ksh and tcsh in a disposable guest and run the expressions. Inspect /etc/shells and check a Bash script with bash -n before running it.

Command or configuration example

```
sudo apt install -y ksh tcsh
bash -c 'course="System Administration"; echo "$course"'
sh -c 'course="System Administration"; echo "$course"'
ksh -c 'course="System Administration"; print -- "$course"'
tcsh -c 'set course = "System Administration"; echo "$course"'
cat /etc/shells
bash -n backup_lab.sh
```

Expected model result. The model prints the same text with shell-specific assignment syntax. bash -n checks syntax but does not execute a backup or establish runtime correctness.

S26 SIMULATION | Educational model

```
Illustrative values and expected interaction; not student execution evidence.
SIMULATED SHELL COMPARISON
Shell   Assignment form                      Result
bash    course="System Administration"        System Administration
sh      course="System Administration"        System Administration
ksh     course="System Administration"        System Administration
tcsh    set course = "System Administration"  System Administration
$ bash -n backup_lab.sh
[No output: model syntax check passes]
Bourne family: sh, bash, ksh
C-shell family: csh, tcsh
```

Simulation figure 26. Compare shell families and portability. Educational model.

PART B | UPDATED COVERAGE

Practical coverage after supplementation

The earlier evidence gaps now have illustrated model applications. Educational coverage is extended; the status of student-executed work remains unchanged.

Lecture	Previous gap	Model coverage
1	Automated users	S01
1	Added hardware and compatibility tests	S02 and S03
1	Documentation workflow and security review	S04 and S05
2	VM, media and installer choices	S06 to S08
2	Partition layout and filesystem shrinking	S09 and S10
2	GRUB, dual boot and boot authentication	S11 and S12
2	User policy, network, time and firewall	S13 to S15
2	Updates, first boot, reboot and shutdown	S16 and S17
2	HTTP, NFS, FTP and UEFI PXE concepts	S18 and S19
2	Autoinstall and golden-image cloning	S20 and S21
3	man -K, less, console and remote X application	S22, S23 and S25
4	Line editing, completion and shell families	S24 and S26

Combined conclusion

Part A demonstrates the observed Ubuntu-session outcomes. Part B makes the remaining practical topics concrete through steps, commands and expected interactions. A real implementation still needs its own verification evidence; model figures are educational examples.

PART B | REFERENCES

Technical references for the models

Ubuntu basic installation
Online documentation

Ubuntu firewall
Online documentation

Ubuntu UEFI PXE
Online documentation

Ubuntu autoinstall reference
Online documentation

Ubuntu autoinstall VM quick start
Online documentation

GNU GRUB manual
Online documentation

OpenSSH and X11 forwarding
Online documentation

man command reference
Online documentation

Source use

The lecture files define the topic coverage; vendor references support the command patterns. Terminal-style figures were prepared for the report and are labelled as models. They do not represent screenshots of the student computer.

Historical examples

Fedora 15 sources, NFS and FTP repositories, Kickstart and Windows deployment tools are categories in the older lecture. The main model uses Ubuntu. Protocol availability probes and platform comparisons do not assert identical support in every modern installer.